# Predicting Asteroid Diameter Based on Photometric Parameters and Orbital Characteristics

Asteroid Project Repo
Christopher Doupis
October 11, 2026

## AI USE DISCLAIMER & ACADEMIC INTEGRITY COMPLIANCE

In compliance with the **National University Student Guidelines for Generative AI**, this project discloses the following tool usage:

- **Project Framing & Query Logic (No Citation Required): Google Gemini** and **OpenAI ChatGPT** were used to brainstorm the machine learning regression scope and mathematically define baseline query parameters for filtering the JPL NASA Small-Body Dataset.
- **Technical Execution, Architecture & EDA (AI-Executed): Google Gemini** acted as the primary execution engine for the programmatic pipeline, packages selection, Exploratory Data Analysis (EDA) setup, and visualization code blocks, which were refactored through interactive human-in-the-loop feedback.
- **Narrative Drafting & Structural Editing (AI-Assisted): Google Gemini** and **OpenAI ChatGPT** served as copywriting and editing assistants to polish language flow, phrasing, and technical explanations of statistics and regression plots across all main report sections.
- **Human Oversight, Validation Loops & External Verification (Author-Owned):** The author maintained complete conceptual control, process oversight, and ownership of the core scientific scope. All AI-generated configurations, code logic, and mathematical interpretations were systematically validated against external peer-reviewed literature and verified third-party sources.
- **Creative Material & Assets (AI-Generated): Midjourney** was utilized for illustrative visual enhancement only (not empirical data), and introductory narrative placeholders contain explicitly marked, author-modified flavor text blocks generated by **OpenAI ChatGPT**.

- **Disclosure Statement (AI-Generated): Google Gemini** was utilized to structure and audit this text format to fulfill all institutional reporting standards.

## REFERENCES

### Peer Reviewed and Scientific References


- **Bora, C. A., Kushvah, B. S., Mouli, G. C., & Yousuf, S.** (2024). Temporal trends in asteroid behaviour: a machine learning and N-body integration approach. Monthly Notices of the Royal Astronomical Society, 534(1), 415–430. https://doi.org/10.1093/mnras/stae2083
- **Ge, J., Zhang, X., Li, J., Zhang, Y., & Jiang, X.** (2026). Asteroid Albedo and Size Inversion: A Deep Learning Method and Application Based on Multiparameters. The Astrophysical Journal Supplement Series, 285(1), 24. https://doi.org/10.3847/1538-4365/ae74b4
- **Ge, J., Zhang, X., Li, J., Wang, H., Xu, D., & Jiang, X.** (2025). Asteroid Types, Albedos, and Diameters Catalog from Gaia DR3: Intelligent Inversion Results via Multisource Information Fusion. The Astrophysical Journal Supplement Series, 280(1), 17. https://doi.org/10.3847/1538-4365/adefe1
- **Murcia, J., Molina, N., Gámez López, M. E., Méndez, N., Mafla, E., & Delgado-Correal, C.** (2023). Optimization of Gauss Method to describe with most accuracy the orbits of Near Earth Asteroids - NEAs. Proceedings of the International Astronomical Union, 19(S374), 15–23. https://doi.org/10.1017/s1743921324000772
- **NASA Office of Inspector General NASA's Implementation and Management of Its Planetary Defense Strategy.** (2025). https://oig.nasa.gov/wp-content/uploads/2025/06/final-report-ig-25-006-nasas-implementation-and-management-of-its-planetary-defense-strategy.pdf
- **Robinson, J. E., Fitzsimmons, A., Young, D. R., Bannister, M., Denneau, L., Erasmus, N., Lawrence, A., Siverd, R. J., & Tonry, J.** (2024). Main-belt and Trojan asteroid phase curves from the ATLAS survey. Monthly Notices of the Royal Astronomical Society, 531(1), 304–326. https://doi.org/10.1093/mnras/stae966
- **Shams, F. A., Nafiz, A. A. M., Mohosheu, Md. S., Hoque, M. M., Abir, S. R., Ratul, R. H., Mushfique, Md. M. R., Nazim, A. Ibn, Khan, R. R., Mahmudunnobe, M., & Kabir, M.** (2025). Asteroid family classification with machine learning: Investigative analysis of a novel two-step approach for categorizing known small asteroid families⋆. Experimental Astronomy, 59(1). https://doi.org/10.1007/s10686-025-09982-y
- **Williams, C.** (2026). Space mining and scientific knowledge of asteroids: Implications for policy in a context of national interest. Space Policy, 101785. https://doi.org/10.1016/j.spacepol.2026.101785

### Generative AI & Software Tools


- **Google.** (2026). *Gemini* [Large language model]. https://google.com
- **Midjourney.** (2026). *Midjourney* [Image generation software]. https://midjourney.com
- **National Aeronautics and Space Administration (NASA) / Jet Propulsion Laboratory (JPL). (n.d.). JPL Small-Body Database Search Engine.** Retrieved from https://ssd.jpl.nasa.gov/tools/sbdb_query.html
- **OpenAI.** (2026). *ChatGPT* [Large language model]. https://openai.com


⚠️ **IMPORTANT:** The term `XGBOOST` is used to refer to `Gradient Boosting Regression` ( `GBR` ) throughout the framework. This is to limit code refactoring and re-running training to replace `XGBOOST` with `GBR` .

# Problem Statement & Hypothesis Formulation

## I. PROBLEM STATEMENT

`Asteroid` `diameter` prediction methods:

- **(1)** using a combination of `very high data collection cost` , `very high predictive power` , and `low observational reliability` features like `albedo` (Group 3) with `low data collection cost` , `medium observational reliability` features, like `absolute magnitude` ( `H` ) (Group 2)
- **(2)** using a combination of `low data collection cost` , `high observational reliability` features (Group 1) with `low data collection cost` , `medium observational reliability` features, like `absolute magnitude` ( `H` ) (Group 2)(Ge, 2026; Murcia, 2023; Robinson, 2024).

The project looks to determine if the `diameter` of an `asteroid` can accurately be predicted with only Group 1 features to maintain the `low data collection cost` , `high observational reliability` of features that are inexpensive and not variable or prone to error while observing.

## II. HYPOTHESIS FORMULATION

The project utilizes a two-model comparison framework that will be assessed from a statistical and operational perspective. The regression models `target` `diameter` ( `Y` ) using two feature subsets: an expanded set of `low data collection`

`cost`, `high observational reliability` observations with `low data collection cost`, `medium observational reliability` observations ($\mathbf{X}_{full}$) and a restricted set of only `low data collection cost`, `high observational reliability` observations ($\mathbf{X}_{restricted}$). The restricted model ($\mathbf{X}_{restricted}$) is preferred due to the lower number and higher realiability of features. Current peer-reviewed research dictates that `absolute magnitude` (`H`), a Group 2 feature, is mathematically required to calculate `diameter` (Ge, 2026).

The hypothesis determines if `asteroid diameter` (`Y`) can be accurately predicted within a statistical and operational threshold ($\delta$ = 0.0500) by the restrictive feature subset compared to the full feature subset, quantified as:

$$\Delta\ R^2_{test} = R^2_{Model(\mathbf{X}_{full})} - R^2_{Model(\mathbf{X}_{restricted})}$$

**Statistical Hypothesis**
The statistical hypothesis tests if the restrictive feature subset of Group 1 features maintains a statistically acceptable degree of accuracy in predicting `target diameter` (`Y`) relative to the expanded feature subset of Group 1 and Group 2 features.

- **Statistical Null Hypothesis ($H_{0,stat}$):** $\Delta\ R^2_{test} \leq 0$

  The inclusion of Group 2 features yield no improvement or a decrease in the out-of-sample $\Delta\ R^2_{test}$, indicating that the expanded features **do not** increase the accuracy of predicting the `target diameter` (`Y`).
- **Statistical Alternative Hypothesis ($H_{1,stat}$):** $\Delta\ R^2_{test} > 0$

  The inclusion of Group 2 features yields an increase in the out-of-sample $\Delta\ R^2_{test}$, indicating that the expanded features **do** increase the accuracy of predicting the `target diameter` (`Y`).

**Operational Hypothesis**
The operational hypothesis tests if the restrictive feature subset of Group 1 features maintains an operationally and practically acceptable degree of accuracy ($\delta$) in predicting `target diameter` (`Y`) relative to the expanded feature subset of Group 1 and Group 2 features.

- **Operational Null Hypothesis ($H_{0,ops}$):** $\Delta\ R^2_{test} > \delta$

  The inclusion of Group 2 features yield an operationally significant increase in accuracy to predict `target diameter` (`Y`), indicating that the restricted feature subset **is not** sufficient enough for operational use.

- **Operational Alternative Hypothesis ($H_{1,ops}$):** $\Delta R^2_{test} \leq \delta$
  The inclusion of Group 2 features does not yields an operationally significant increase in accuracy to predict `target diameter` (`Y`), indicating that the restricted feature subset **is** sufficient enough for operational use.

# The Data Science Process

## I. ACQUIRE

The JPL NASA Small-Body Dataset was chosen for the following reasons:

- Contains Group 1 and Group 2 characteristics such as `absolute magnitude` (`H`), `semi-major axis` (`a`), `eccentricity` (`e`), and `inclination` (`i`) which directly relate to the problem statement and hypothesis requirements necessary for the expanded features subset and restricted feature subset.
- NASA is an authoritative, reliable, and primary repository of small-body data that is also open-source and available to the public allowing reproduction of testing for peer-review.

The JPL NASA Small-Body Dataset was queried to meet `target diameter` (`Y`) to be used in `out-of-sample` testing. The query exclusively included `asteroids` with a known `diameter` greater than zero kilometers (`0 km`).

## II. PREPARE

Data has been prepared for regression analysis in the following ways:

1. **Data Structure Integrity:** Validate all data types are correct. Name columns should be `strings` (`str`), continuous values should be `floats` (`float64`), and discrete values should be `integers` (`int`).
2. **Data Domain Validation:** Remove data is implausible such as negative `diameter`.
3. **Duplicate Data Removal:** Remove duplicate rows of data and observations of the same small-body. Multiple observations of the same small-body skew the sampling data in regression and create a data leakage problem with artificial accuracy in the case where an small-body observation ends up in the training and testing datasets.
4. **Target Variable Isolation (diameter):** All entries without a `diameter` have been removed. `Diameter` (`Y`) is the `target` variable and is required for the training model and to calculate the coefficient of determination *($R^2_{test}$)* during `out-of-sample` testing.

5. **Exclusion of High-Cost Features (albedo):** `Albedo` represents a Group 3 characteristic and was explicitly dropped from the dataset. Removing this feature ensures the model does not rely on cost-prohibited data and maintains operational cost-effective analysis.
6. **Feature Subset Selection:** The model requires the `target` `diameter` and only features that fall into Group 1 *(e.g.,* `semi-major axis` `a`, `eccentricity` `e`, `inclination` `i`*)* and Group 2 *(* `absolute magnitude` `H`*)*.
7. **Feature Completeness:** Entries with missing values in the selected feature subsets were removed. This ensures the regression algorithms receive a complete feature matrix and that any observed variance in $R^2_{test}$ is attributable to specific feature subsets and not fluctuations in sample size or missing data.
8. **Target Variable Distribution Scaling:** Scale `diameter` (`Y`) using a `base-10` `logarithmic` formula. Small-bodies size distribution is non-linear. Scaling the `diameter` (`Y`) ensures that the model weights small and large small-bodies the same.
9. **Outlier Detection:** Because outliers represent a real small-body and not an entry error or out-of-scope feature, outliers will be reviewed but may not be removed just on the nature of being an outlier.
10. **Feature Independence:** Remove any highly correlated features to reduce the risk of improper feature ranking between the correlated features.

In [1]:
```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import plotly.express as px
from sklearn.linear_model import LinearRegression
import yaml

with open('config.yaml', 'r') as f:
    config = yaml.safe_load(f)

dataset_path = config['paths']['dataset']
dataset_cleaned_path = config['paths']['dataset_cleaned']
x_restricted = config['features']['x_restricted']
x_full = config['features']['x_full']
y = config['features']['target']

print("=" * 50)
print("INITIALIZING DEFAULT PARAMETERS:")
print("=" * 50)
```

```
print('raw datset: ', dataset_path)
print('cleaned datset: ', dataset_cleaned_path)
print('restricted features: ', x_restricted)
print('full features: ', x_full)
print('target: ', y)
print("=" * 50)

df = pd.read_csv(dataset_path)
df.head()
```

```
==================================================
INITIALIZING DEFAULT PARAMETERS:
==================================================
raw datset:  data/dataset-jpl-small-body-asteroids.csv
cleaned datset:  data/cleaned-dataset-jpl-small-body-asteroids.csv
restricted features:  ['a', 'e', 'i']
full features:  ['a', 'e', 'i', 'H']
target:  diameter
==================================================
```

Out[1]:

| | spkid | diameter | albedo | H | full_name | a | e | i | om | w | q | ad | per_y | data_arc | co |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **0** | 20000001 | 939.400 | 0.0900 | 3.34 | 1 Ceres (A801 AA) | 2.766 | 0.0797 | 10.59 | 80.25 | 73.29 | 2.545 | 2.99 | 4.60 | 9520.0 | |
| **1** | 20000002 | 513.000 | 0.1550 | 4.12 | 2 Pallas (A802 FA) | 2.770 | 0.2307 | 34.93 | 172.89 | 310.97 | 2.131 | 3.41 | 4.61 | 80828.0 | |
| **2** | 20000003 | 246.596 | 0.2140 | 5.19 | 3 Juno (A804 RA) | 2.671 | 0.2557 | 12.99 | 169.81 | 247.90 | 1.988 | 3.35 | 4.37 | 80897.0 | |
| **3** | 20000004 | 522.770 | 0.4228 | 3.25 | 4 Vesta (A807 FA) | 2.361 | 0.0902 | 7.14 | 103.70 | 151.47 | 2.148 | 2.57 | 3.63 | 25743.0 | |
| **4** | 20000005 | 106.699 | 0.2740 | 6.95 | 5 Astraea (A845 XA) | 2.577 | 0.1876 | 5.36 | 141.45 | 359.38 | 2.093 | 3.06 | 4.14 | 65687.0 | |

## 1. Data Structure Integrity

All features are the correct data type. Discrete values like `skpid` (the small-body unique identifier) and `condition code` are integers *( `int` )* and all numerical measurements and values like `a` , `e` , and `i` are floats *( `float64` )*.

In [2]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 139582 entries, 0 to 139581
Data columns (total 18 columns):
 #   Column          Non-Null Count   Dtype  
---  ------          --------------   -----  
 0   spkid           139582 non-null  int64  
 1   diameter        139582 non-null  float64
 2   albedo          138459 non-null  float64
 3   H               139203 non-null  float64
 4   full_name       139582 non-null  str    
 5   a               139582 non-null  float64
 6   e               139582 non-null  float64
 7   i               139582 non-null  float64
 8   om              139582 non-null  float64
 9   w               139582 non-null  float64
 10  q               139582 non-null  float64
 11  ad              139582 non-null  float64
 12  per_y           139582 non-null  float64
 13  data_arc        139577 non-null  float64
 14  condition_code  139582 non-null  int64  
 15  n_obs_used      139582 non-null  int64  
 16  n_del_obs_used  243 non-null     float64
 17  n_dop_obs_used  243 non-null     float64
dtypes: float64(14), int64(3), str(1)
memory usage: 19.2 MB
```

## 2. Data Domain Validation

Removed data that is implausible. Since the data is not validated by an astrophysicist, the project will be limited to checking just the `diameter` (`Y`). Diameters are all valid because `(df['diameter'] <= 0).sum() = 0`.

In [3]:
```
print(f"Number of invalid diameter records: {(df['diameter'] <= 0).sum()}")
```

```
Number of invalid diameter records: 0
```

## 3. Duplicate Data Removal

Removed rows with identical values in every column and where rows share the same `skpid`. Confirmed removed when `df.duplicated().sum() = 0` and `df.duplicated(subset=['spkid']).sum() = 0`.

In [4]:
```
print("# of rows with identical values in every single column: ", df.duplicated().sum())
print("# of rows with the same skpid", df.duplicated(subset=['spkid']).sum())
```

```
# of rows with identical values in every single column:  0
# of rows with the same skpid 0
```

### 4. Target Variable Isolation

All entries without a `diameter` (`Y`) have been removed. Confirmed because `.isnull().sum() = 0` for `diameter` (`Y`).

### 5. Exclusion of High-Cost Features

Drop `albedo` because it is a high-cost observation that will not be included in the model. Confirmed removed because feature does not show up in `.info()` or `.isnull().sum()`.

In [5]:
```
df_clean = df.dropna(subset=y).drop(columns=['albedo'])
df_clean.isnull().sum()
```

Out[5]:
```
spkid                0
diameter             0
H                  379
full_name            0
a                    0
e                    0
i                    0
om                   0
w                    0
q                    0
ad                   0
per_y                0
data_arc             5
condition_code       0
n_obs_used           0
n_del_obs_used    139339
n_dop_obs_used    139339
dtype: int64
```

## 6. Feature Subset Selection

Removed all features that are not part of the full feature subset because they are metadata or not orbital characteristics used to determine `diameter` (`Y`).

In [6]:
```
df_clean = df_clean[[y] + x_full + ['ad', 'q', 'om', 'w']]
df_clean.head()
```

Out[6]:

| | diameter | a | e | i | H | ad | q | om | w |
|---|---|---|---|---|---|---|---|---|---|
| **0** | 939.400 | 2.766 | 0.0797 | 10.59 | 3.34 | 2.99 | 2.545 | 80.25 | 73.29 |
| **1** | 513.000 | 2.770 | 0.2307 | 34.93 | 4.12 | 3.41 | 2.131 | 172.89 | 310.97 |
| **2** | 246.596 | 2.671 | 0.2557 | 12.99 | 5.19 | 3.35 | 1.988 | 169.81 | 247.90 |
| **3** | 522.770 | 2.361 | 0.0902 | 7.14 | 3.25 | 2.57 | 2.148 | 103.70 | 151.47 |
| **4** | 106.699 | 2.577 | 0.1876 | 5.36 | 6.95 | 3.06 | 2.093 | 141.45 | 359.38 |

## 7. Feature Completeness

Checked for any missing or null values using using `.isna().sum()`. If missing values, removed the entire entry using `.dropna()`. Since the only feature with missing values is `H` the entries can be deleted without review. This is because `H` is the feature difference in the hypothesis and is fundamental and required to test the model. For the same reasons, it is not appropriate to estimate this value.

In [7]:
```
df_clean.isna().sum()
```

Out[7]:
```
diameter      0
a             0
e             0
i             0
H           379
ad            0
q             0
om            0
w             0
dtype: int64
```

In [8]:
```
df_clean.dropna(subset=['H'], inplace=True)
df_clean.isna().sum()
```

Out[8]:
```
diameter    0
a           0
e           0
i           0
H           0
ad          0
q           0
om          0
w           0
dtype: int64
```

## 8. Target Variable Distribution Scaling

Verified `target` `diameter` skew is greater than one (`1`). Since it was highly skewed, the `diameter` (`Y`) was transformed into a `log` `base-10` scale, `diameter_log`, so the regression model does not improperly weight errors between small-body sizes. After the transformation, `diameter_log` was checked for errors in values and the entries were removed as necessary.

In [9]:
```
df_clean['diameter'].describe()
```

Out[9]:
```
count    139203.000000
mean          5.469118
std           9.318406
min           0.002500
25%           2.773000
50%           3.957000
75%           5.739000
max         939.400000
Name: diameter, dtype: float64
```

In [10]:
```
print(df_clean['diameter'].skew())
```

```
25.930498594815756
```

In [11]:
```
df_clean['diameter_log'] = np.log10(df_clean['diameter'])
print(f"Total infinity anomalies found: {np.isinf(df_clean['diameter_log']).sum()}")
print(f"Total na values found: {df_clean['diameter_log'].isna().sum()}")
df_clean['diameter_log'].describe()
```

```
Total infinity anomalies found: 0
Total na values found: 0
```

Out[11]:
```
count    139203.000000
mean          0.615454
std           0.282694
min          -2.602060
25%           0.442950
50%           0.597366
75%           0.758836
max           2.972851
Name: diameter_log, dtype: float64
```

## 9. Outlier Detection

Calculated the lower and upper bounds from the `interquartile range` ( `IQR` ) and reviewed all outliers. No entries were determiend to be errors so outliers were kept in the dataset because they represent a real small-body that is part of the population of data. The `diameter` was reviewed to compare to `diameter_log` but will not be used in the model.

In [12]:
```
q1 = df_clean.quantile(0.25)
q3 = df_clean.quantile(0.75)
iqr = q3 - q1

lower_bound = q1 - 1.5 * iqr
upper_bound = q3 + 1.5 * iqr

total_rows = len(df_clean)

for col in df_clean.select_dtypes(include=['number']).columns:
    low_count = (df_clean[col] < lower_bound[col]).sum()
    high_count = (df_clean[col] > upper_bound[col]).sum()

    low_pct = (low_count / total_rows) * 100
    high_pct = (high_count / total_rows) * 100

    print(f"{col} -> Lower Outliers: {low_count} ({low_pct:.2f}%) | Upper Outliers: {high_count} ({high_pc
```

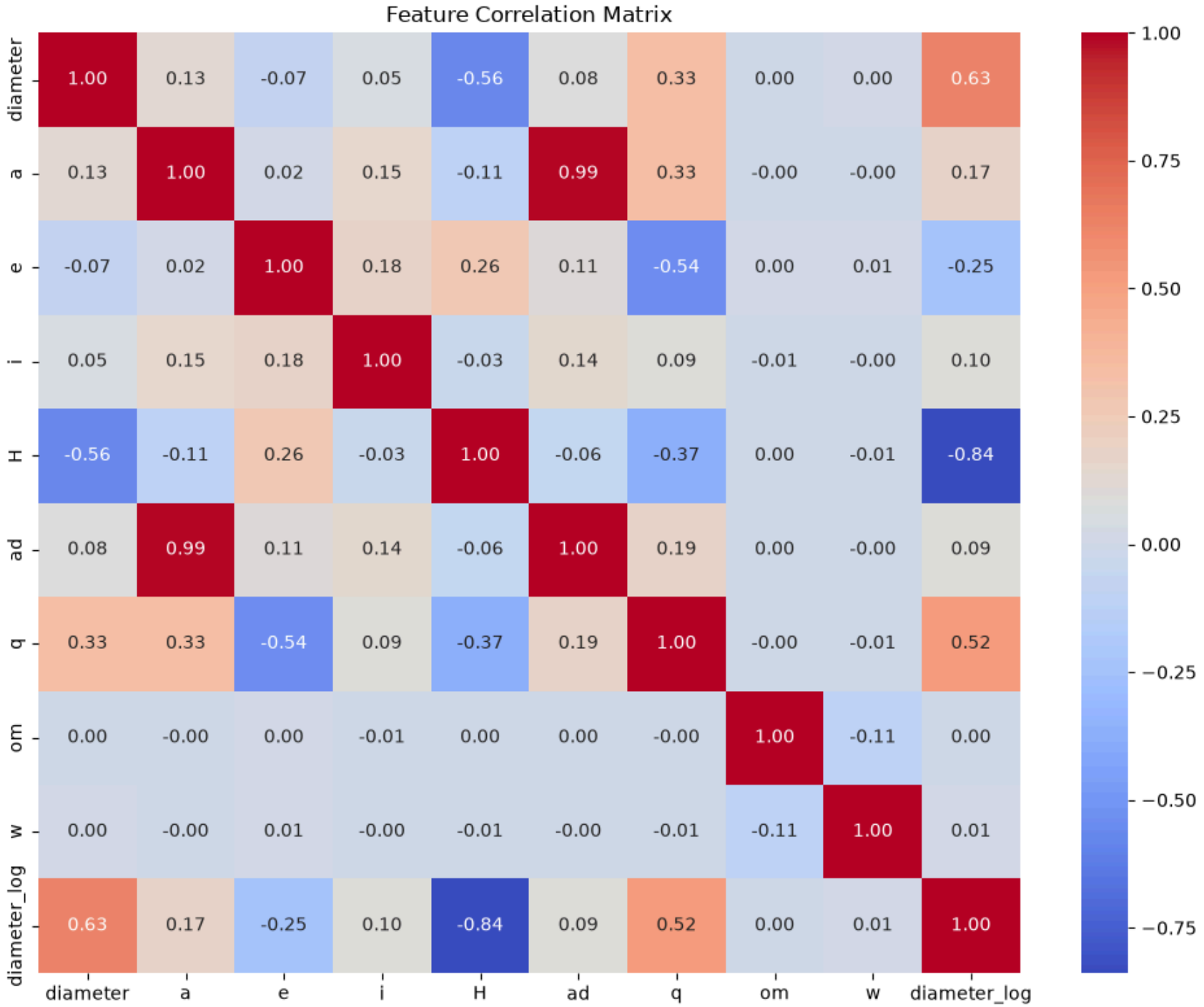
```
diameter -> Lower Outliers: 0 (0.00%) | Upper Outliers: 9809 (7.05%)
a -> Lower Outliers: 579 (0.42%) | Upper Outliers: 3060 (2.20%)
e -> Lower Outliers: 0 (0.00%) | Upper Outliers: 1886 (1.35%)
i -> Lower Outliers: 0 (0.00%) | Upper Outliers: 4322 (3.10%)
H -> Lower Outliers: 3075 (2.21%) | Upper Outliers: 815 (0.59%)
ad -> Lower Outliers: 449 (0.32%) | Upper Outliers: 3365 (2.42%)
q -> Lower Outliers: 1091 (0.78%) | Upper Outliers: 1983 (1.42%)
om -> Lower Outliers: 0 (0.00%) | Upper Outliers: 0 (0.00%)
w -> Lower Outliers: 0 (0.00%) | Upper Outliers: 0 (0.00%)
diameter_log -> Lower Outliers: 840 (0.60%) | Upper Outliers: 3510 (2.52%)
```

### 10. Feature Independence

Created a `heatmap` to review highly correlated features. `a` and `ad` are highly correlated. Since `ad` is derived from `a` as `ad = a x (1 + e)` it will be removed so it does not get improperly weighted in the regression analysis. Features `om` and `w` have no feature-to-feature correlation or feature-to-target correlation. These will be kept since tree-based regression can factor these out if they are unnecessary.

In [13]:
```
# create correlation matrix to see if any variables inherently correlate to diameter
plt.figure(figsize=(10,8))
sns.heatmap(df_clean.corr(numeric_only=True), annot=True, cmap='coolwarm', fmt=".2f")
plt.title("Feature Correlation Matrix")
```

```
plt.tight_layout()
plt.savefig("figures/feature_correlation_matrix.png", dpi=300)
plt.show()
```


Feature Correlation Matrix
diameter
a
e
i
H
ad
q
om
w
diameter_log
1.00 0.13 -0.07 0.05 -0.56 0.08 0.33 0.00 0.00 0.63
0.13 1.00 0.02 0.15 -0.11 0.99 0.33 -0.00 -0.00 0.17
-0.07 0.02 1.00 0.18 0.26 0.11 -0.54 0.00 0.01 -0.25
0.05 0.15 0.18 1.00 -0.03 0.14 0.09 -0.01 -0.00 0.10
-0.56 -0.11 0.26 -0.03 1.00 -0.06 -0.37 0.00 -0.01 -0.84
0.08 0.99 0.11 0.14 -0.06 1.00 0.19 0.00 -0.00 0.09
0.33 0.33 -0.54 0.09 -0.37 0.19 1.00 -0.00 -0.01 0.52
0.00 -0.00 0.00 -0.01 0.00 0.00 -0.00 1.00 -0.11 0.00
0.00 -0.00 0.01 -0.00 -0.01 -0.00 -0.01 -0.11 1.00 0.01
0.63 0.17 -0.25 0.10 -0.84 0.09 0.52 0.00 0.01 1.00
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75

In [14]:
```
df_clean = df_clean.drop(columns=['ad'])
df_clean.head()
```

Out[14]:

| | diameter | a | e | i | H | q | om | w | diameter_log |
|---|---|---|---|---|---|---|---|---|---|
| **0** | 939.400 | 2.766 | 0.0797 | 10.59 | 3.34 | 2.545 | 80.25 | 73.29 | 2.972851 |
| **1** | 513.000 | 2.770 | 0.2307 | 34.93 | 4.12 | 2.131 | 172.89 | 310.97 | 2.710117 |
| **2** | 246.596 | 2.671 | 0.2557 | 12.99 | 5.19 | 1.988 | 169.81 | 247.90 | 2.391986 |
| **3** | 522.770 | 2.361 | 0.0902 | 7.14 | 3.25 | 2.148 | 103.70 | 151.47 | 2.718311 |
| **4** | 106.699 | 2.577 | 0.1876 | 5.36 | 6.95 | 2.093 | 141.45 | 359.38 | 2.028160 |

In [15]:
```
# save cleaned data to csv
df_clean.to_csv(dataset_cleaned_path, index=False)
```

## III. ANALYZE (EDA)

Data has been analyzed in the following ways:

1. **Individual Feature Boxplots:** Visualize distribution of individual features using `boxplots`.
2. **Target Normalization Check:** Visualize log transformation of `target` `diameter` (`Y`) using a `histogram`.
3. **Primary Signal Feature:** Visualize `absolute magnitude` (`H`) versus `target` `diatmeter` (`Y`) for correlation using a `scatterplot`.
4. **Orbital Characteristic Features:** Visualize features versus `target` `diatmeter` (`Y`), `absolute magnitude` (`H`), and other features for correlation and inclusion in model using `2D` and `3D` `scatterplots` with `visual mapping`.
5. **Excluded Features:** Determine features that do not require additional analysis.

### 1. Individual Feature Boxplots

Visually reviewed distribution of individual features. Several of the physical features (`a`, `e`, and `i`) are heavily skewed. This indicates `linear regression models` are not ideal and supports utilizing `tree-based models`. `e` (`eccentricity`) represents the orbit of a small-body with `Earth` having an `e ≈ 0.017`. This means the small-body data set has a relative stable orbit similar to `Earth` while some outliers have less stable orbits. `i` represents `inclination` relative to `Earth's`

( `Earth's i = 0°` ). This means the small-body dataset distribution of `i` has a similar inclination as `Earth` which makes sense given they are under the same orbital parameters as `Earth` . `a` represents the `semi-major axis` which is the average distance from the Sun. The distribution of `a` means that most of the small-bodies are in the main asteroid belt between Mars and Jupiter with some outliers as far out as Neptune or farther.

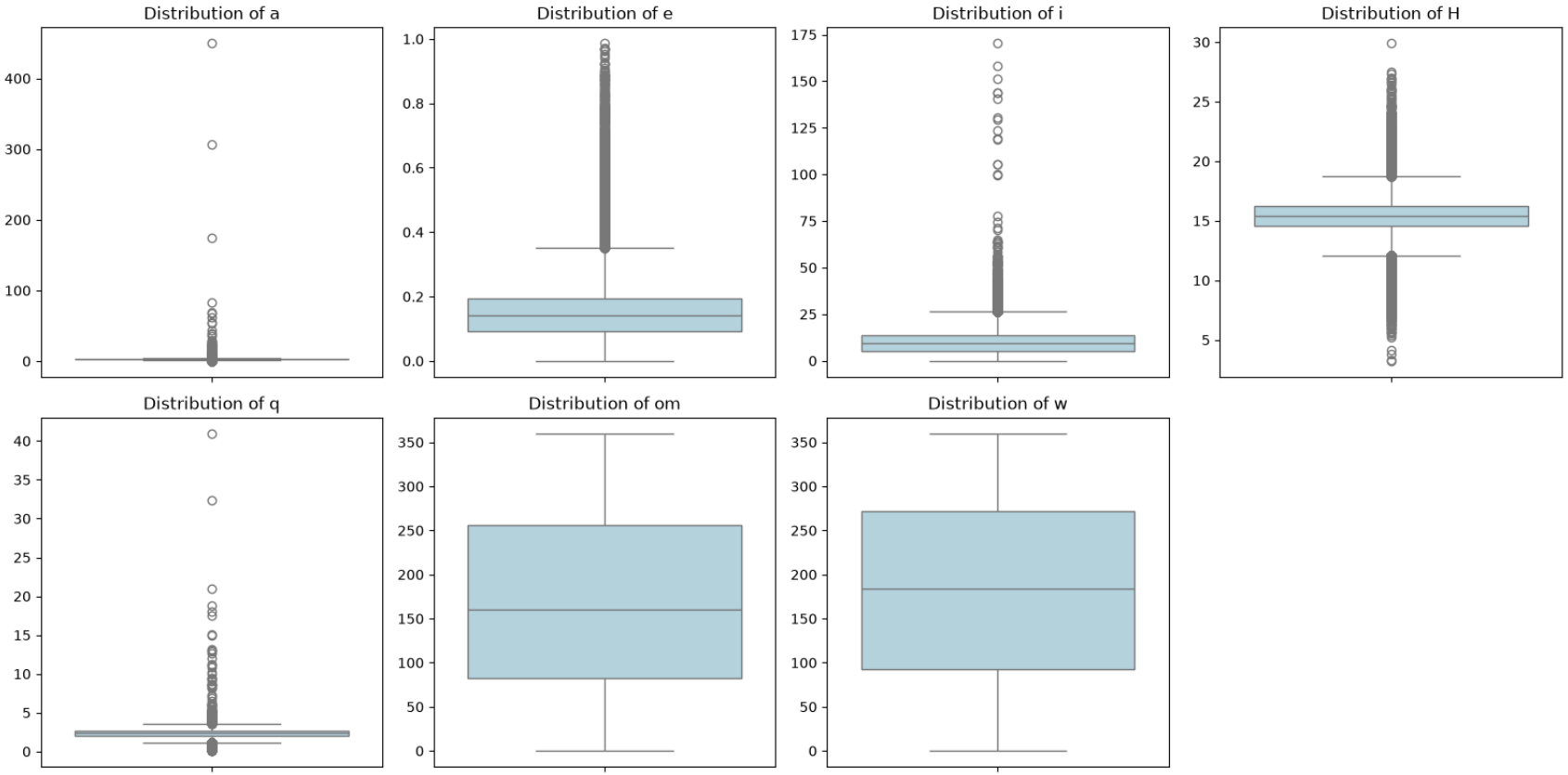
All features have acceptable distributions in context to small-bodies.

In [16]:
```
fig, axes = plt.subplots(2, 4, figsize=(16, 8))
axes = axes.flatten()
plot_features = x_full + ['q', 'om', 'w']

for idx, col in enumerate(plot_features):
    sns.boxplot(data=df_clean, y=col, ax=axes[idx], color='lightblue')
    axes[idx].set_title(f'Distribution of {col}')
    axes[idx].set_ylabel('')

for i in range(len(plot_features), len(axes)):
    fig.delaxes(axes[i])

plt.tight_layout()
plt.savefig("figures/feature_distros.png", dpi=300)
plt.show()
```




## 2. Target Normalization Check

Compared the `diameter` to `diameter_log` to visualize skew. The raw `target` `diameter` is `power-law` skewed to the right. The `logarithmic` transformation decompresses the skew and better prepares the data for the `tree-based model` analysis.

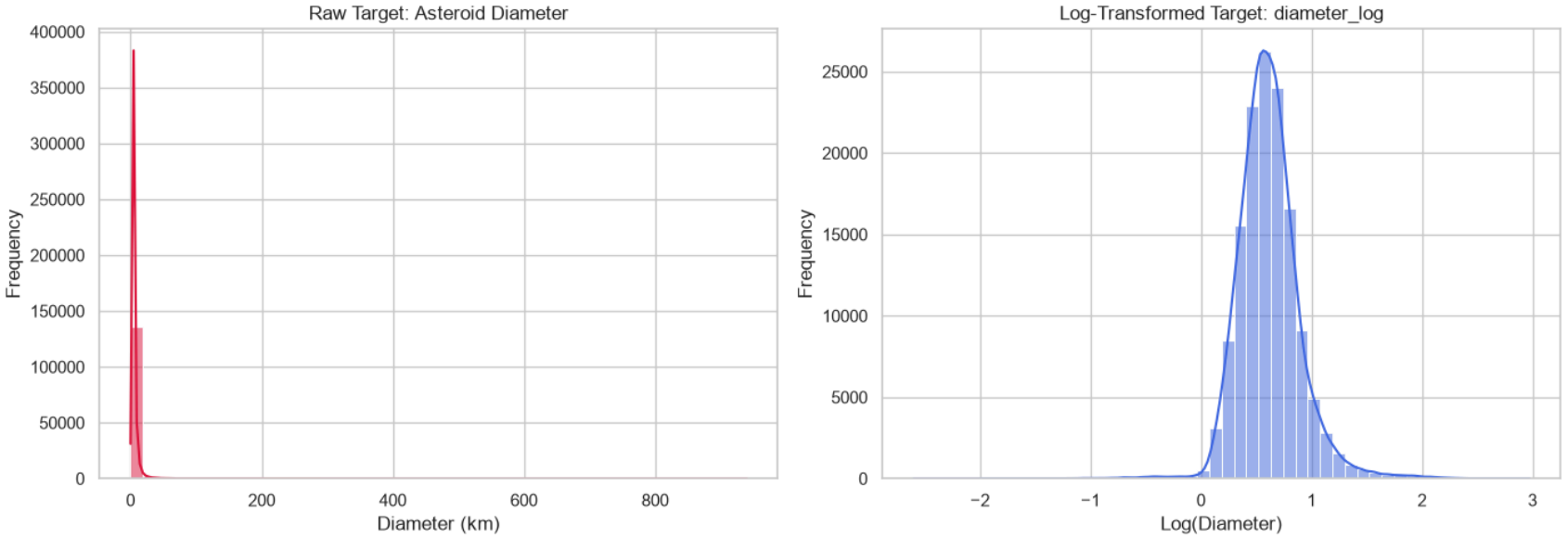
In [17]:
```
sns.set_theme(style="whitegrid")
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

sns.histplot(df_clean['diameter'], bins=50, kde=True, ax=axes[0], color='crimson')
axes[0].set_title('Raw Target: Asteroid Diameter')
axes[0].set_xlabel('Diameter (km)')
axes[0].set_ylabel('Frequency')

sns.histplot(df_clean['diameter_log'], bins=50, kde=True, ax=axes[1], color='royalblue')
```

```
axes[1].set_title('Log-Transformed Target: diameter_log')
axes[1].set_xlabel('Log(Diameter)')
axes[1].set_ylabel('Frequency')

plt.tight_layout()
plt.savefig("figures/target_log_transform.png", dpi=300)
plt.show()
```




## 3. Primary Signal Feature

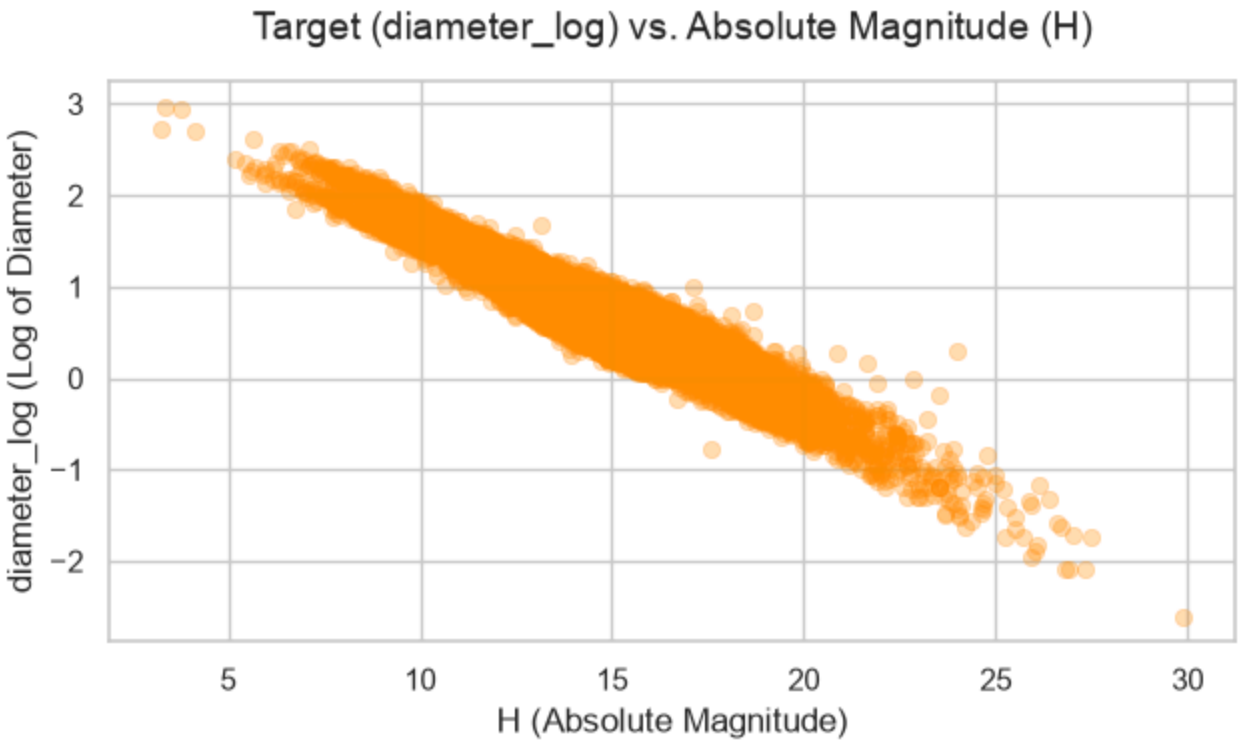
`H` is the signal feature that differentiates the full and restricted feature subsets and is highly correlated to `diameter`. The scatterplot visually confirms this correlation and shows an inverse relationship between `H` and `diameter_log`. `Absolute magnitude` (`H`) represents `intrinsic brightness` of a small-body where a smaller `H` value means higher brightness. This trend shows that a small-body with a higher brightness (lower `H` value) tends to be larger and a small-body with a lower brightness (higher `H` value) tends to be smaller. This visualization supports the `peer-reviewed` research that `H` is highly correlated to `diameter` and important to predicting a small-body `diameter`.

In [18]:
```
sns.set_theme(style="whitegrid")
plt.figure(figsize=(6.5, 4))

sns.scatterplot(data=df_clean, x='H', y='diameter_log', alpha=0.3, color='darkorange', edgecolor=None)
```

```
plt.title('Target (diameter_log) vs. Absolute Magnitude (H)', fontsize=14, pad=15)
plt.xlabel('H (Absolute Magnitude)', fontsize=12)
plt.ylabel('diameter_log (Log of Diameter)', fontsize=12)

plt.tight_layout()
plt.savefig("figures/target_v_abs_mag", dpi=300)
plt.show()
```




## 4. Orbital Characteristic Features

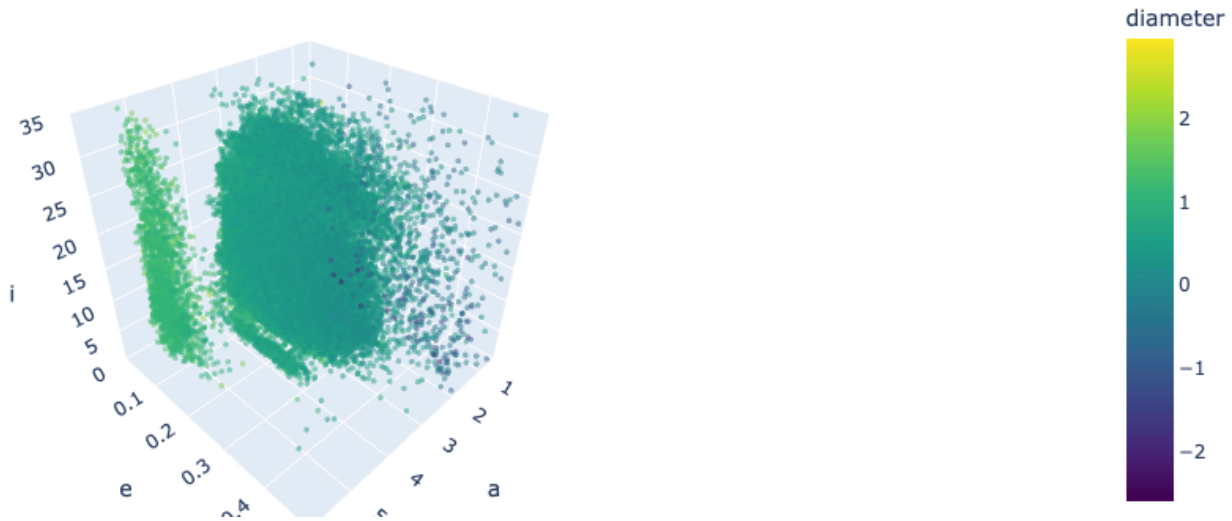
### Fundamental Orbital Elements

Visualizing `a`, `e`, and `i` (independent orbital characteristics) vs. `diameter` to see if there is a correlation between these three features and `diameter`. The visualization reveals distinct clustering demonstrating that certain orbital groupings share diameter profiles. This provides reasonable evidence to use these features as candidates for out-of-sample testing in the model.

In [19]:
```
df_viewport = df_clean[df_clean['a'] <= 10]

fig = px.scatter_3d(
    df_viewport, x='a', y='e', z='i', color='diameter_log',
    color_continuous_scale='viridis', opacity=0.5, title='Orbital Size, Shape, and Tilt vs Diameter',
    labels={'a': 'a', 'e': 'e', 'i': 'i', 'diameter_log': 'diameter'},
)

fig.update_traces(marker=dict(size=2))
fig.update_scenes(xaxis_range=[0.5, 6.0], yaxis_range=[0, 0.5], zaxis_range=[0, 35])
fig.update_layout(margin=dict(l=0, r=0, b=0, t=40))
fig.write_image("figures/features_diameter_mapping.png", scale=3)
fig.show()
```




### Absolute Magnitude (H) and Restricted Subset Fundamental Characteristics

#### Absolute Magnitude (H) versus Individual Restricted Subset Fundamental Characteristics

Visualize `a`, `e`, and `i` individually vs. `H` in terms of `diameter_log`. Visualization shows a clear indication that `H` is a primary indicator for predicting `diameter` but no distinct correlation exists between `H` and any of the individual features.

In [20]:
```
sns.set_theme(style="whitegrid")
fig, axes = plt.subplots(1, 3, figsize=(18, 5))
```

```
sns.scatterplot(data=df_clean, x='a', y='H', hue='diameter_log', palette='viridis', alpha=0.5, s=8, ax=axes
axes[0].set_title('Orbit Size (a) in AU vs. Absolute Magnitude (H)')
axes[0].set_xlabel('a')
axes[0].set_ylabel('H')
axes[0].set_xlim(0.5, 6.0)

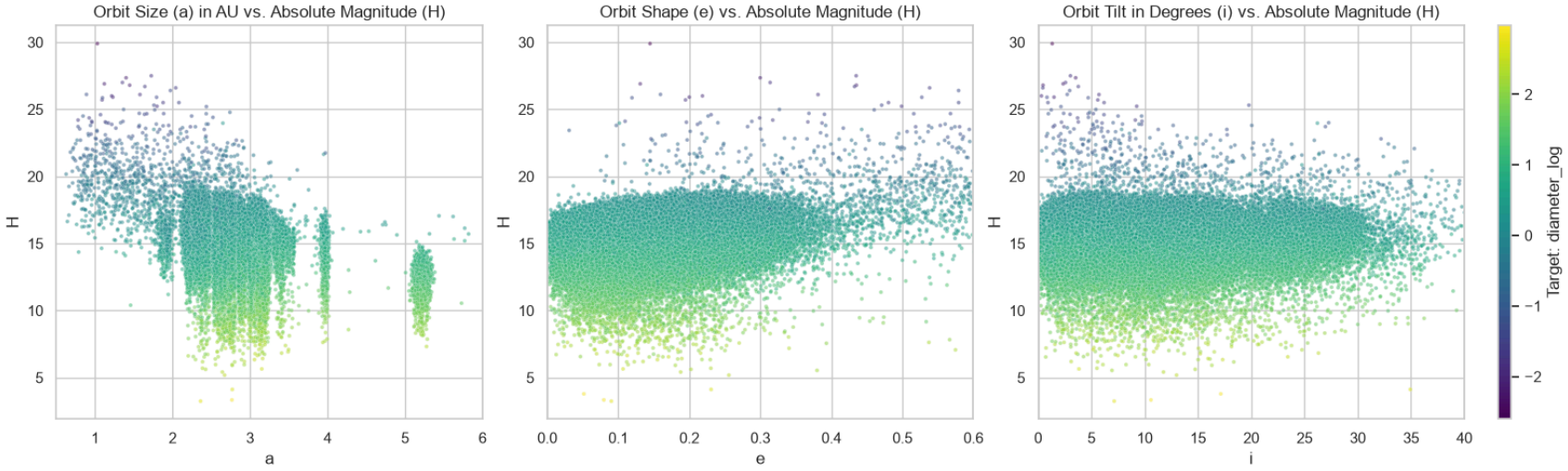
sns.scatterplot(data=df_clean, x='e', y='H', hue='diameter_log', palette='viridis', alpha=0.5, s=8, ax=axes
axes[1].set_title('Orbit Shape (e) vs. Absolute Magnitude (H)')
axes[1].set_xlabel('e')
axes[1].set_ylabel('H')
axes[1].set_xlim(0, 0.6)

sns.scatterplot(data=df_clean, x='i', y='H', hue='diameter_log', palette='viridis', alpha=0.5, s=8, ax=axes
axes[2].set_title('Orbit Tilt in Degrees (i) vs. Absolute Magnitude (H)')
axes[2].set_xlabel('i')
axes[2].set_ylabel('H')
axes[2].set_xlim(0, 40)

axes[0].get_legend().remove()
plt.tight_layout()

norm = plt.Normalize(df_clean['diameter_log'].min(), df_clean['diameter_log'].max())
sm = plt.cm.ScalarMappable(cmap="viridis", norm=norm)
sm.set_array([])
cbar = fig.colorbar(sm, ax=axes.tolist(), orientation='vertical', pad=0.02, aspect=30)
cbar.set_label('Target: diameter_log', fontsize=12)

plt.savefig("figures/features_v_abs_mag", dpi=300)
plt.show()
```




Absolute Magnitude (H) in Relation to Restricted Subset Fundamental Characteristics

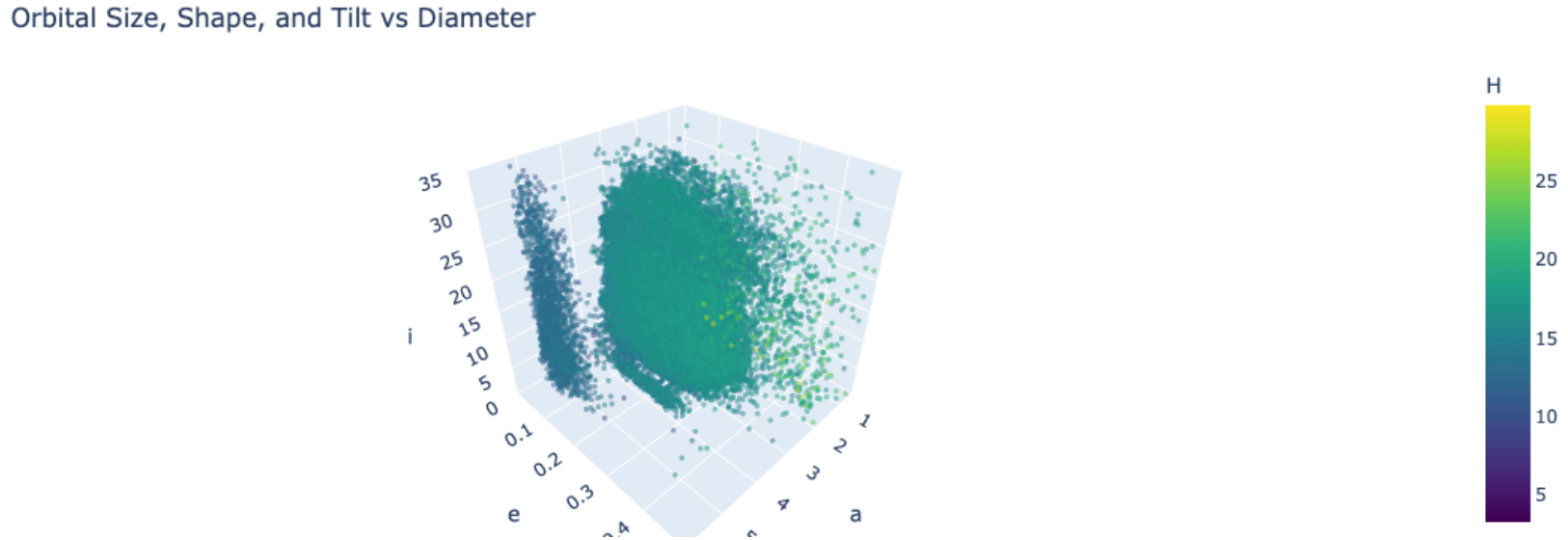
Visualizing `a`, `e`, and `i` (independent orbital characteristics) vs. `absolute magnitude` (`H`) to see if there is a correlation between these three features and `H`. The visualization reveals distinct clustering demonstrating that certain orbital groupings share magnitude profiles. This makes sense since `H` and `diameter` are closely related so the independent orbital characteristics should exhibit a nearly identical structural trend when mapped by `H` as they do when mapped by `diameter`.

In [21]:
```
df_viewport = df_clean[df_clean['a'] <= 10]

fig = px.scatter_3d(
    df_viewport, x='a', y='e', z='i', color='H',
    color_continuous_scale='viridis', opacity=0.5, title='Orbital Size, Shape, and Tilt vs Diameter',
    labels={'a': 'a', 'e': 'e', 'i': 'i', 'H': 'H'},
)

fig.update_traces(marker=dict(size=2))
fig.update_scenes(xaxis_range=[0.5, 6.0], yaxis_range=[0, 0.5], zaxis_range=[0, 35])
fig.update_layout(margin=dict(l=0, r=0, b=0, t=40))

fig.write_image("figures/features_abs_mag_mapping.png", scale=3)
fig.show()
```



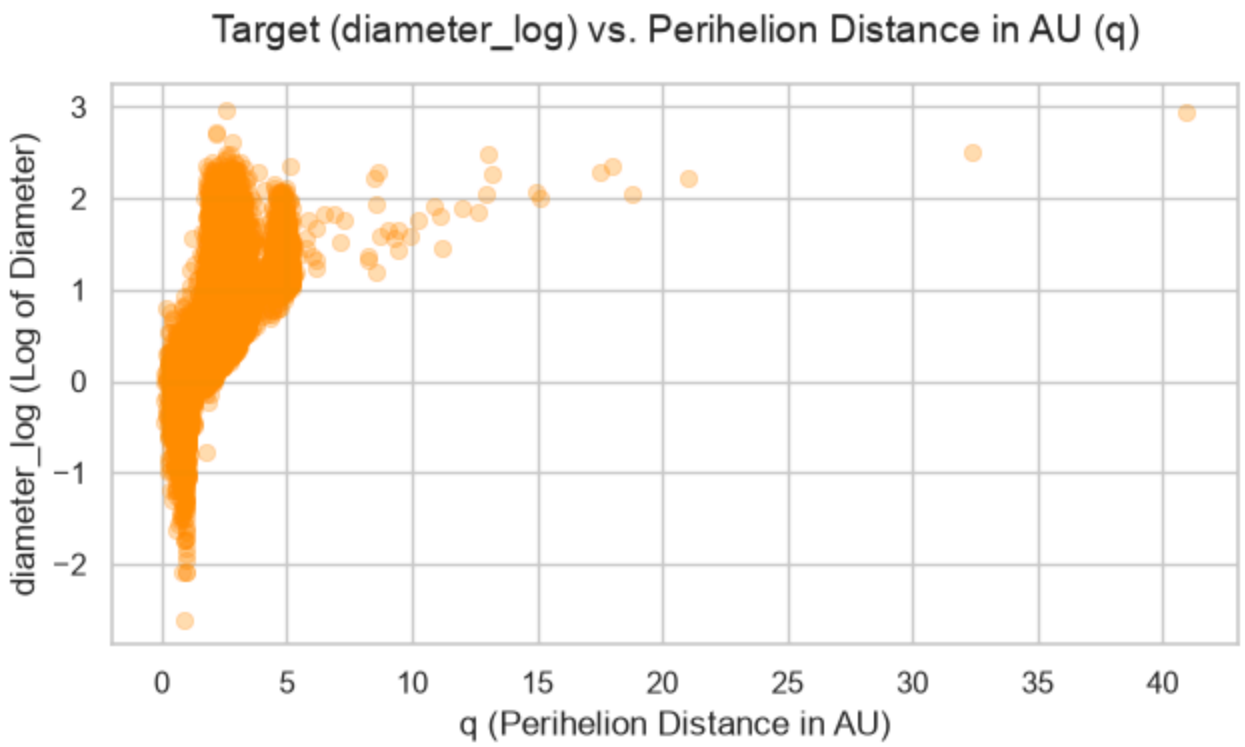

### Derived Orbital Characteristics

`q` is a derived characteristic from `a` and `e` as `q = a(1-e)`. Visualizing `q` vs `diameter_log` shows a trend that the farther from the Sun a small-body gets, the larger it gets. It has a strong `linear slope` specifically at `1 AU` up to `6 AU`. Logically this does not make sense since the size of a small-body is not dictated by the distance from the Sun. There are small and large small-bodies near and far from the Sun. Since this is a `bias` in interpretation and not a flaw in the data, `q` can be kept in the dataset similar to `om` and `w` to be included or excluded in the final model determined at a later step.

In [22]:
```
sns.set_theme(style="whitegrid")
plt.figure(figsize=(6.5, 4))

sns.scatterplot(data=df_clean, x='q', y='diameter_log', alpha=0.3, color='darkorange', edgecolor=None)
plt.title('Target (diameter_log) vs. Perihelion Distance in AU (q)', fontsize=14, pad=15)
plt.xlabel('q (Perihelion Distance in AU)', fontsize=12)
plt.ylabel('diameter_log (Log of Diameter)', fontsize=12)
plt.tight_layout()

plt.savefig("figures/diameter_log_v_peri_dist.png", dpi=300)
plt.show()
```



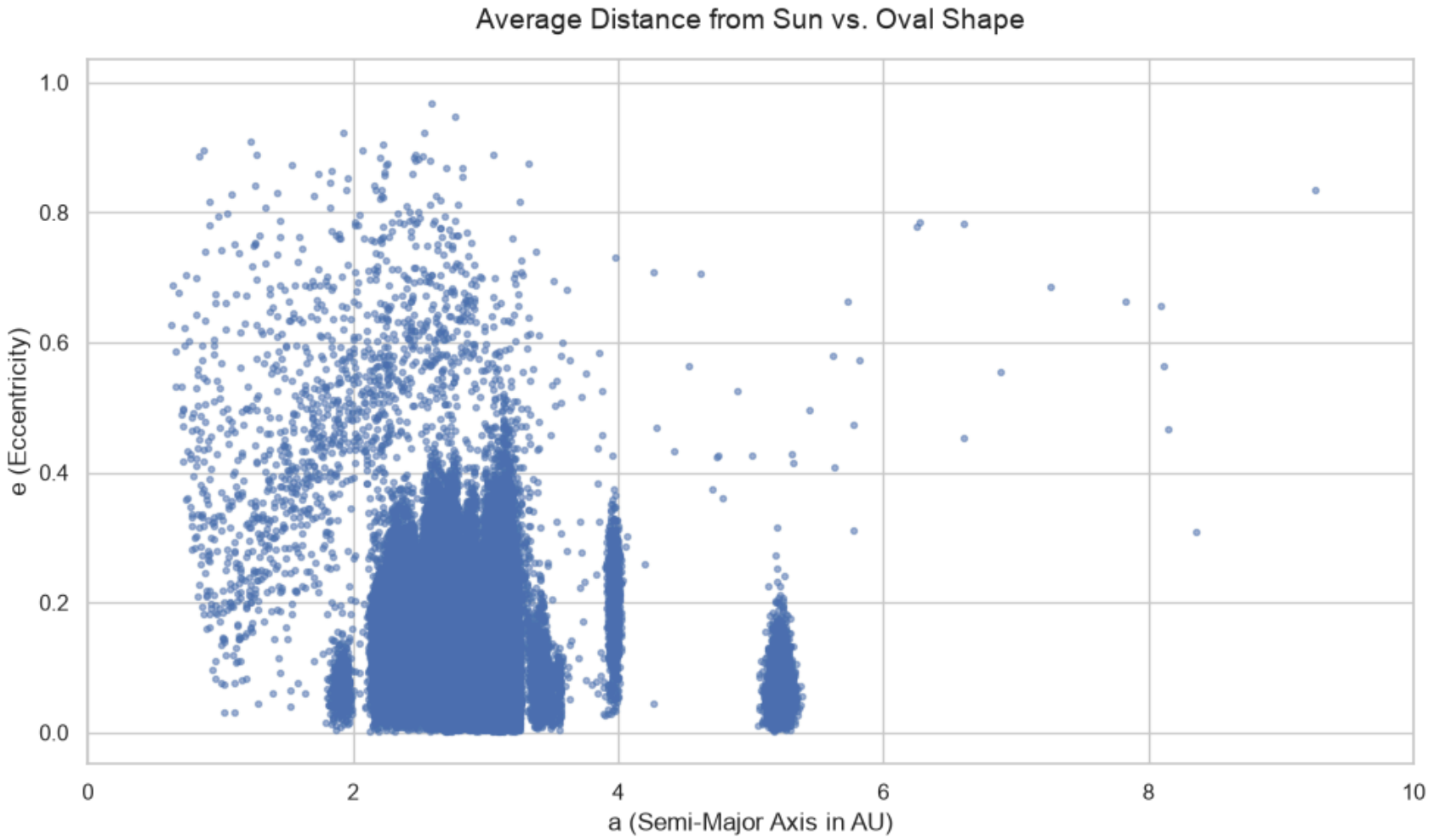

### Semi-Major Axis v. Eccentricity

Visualizing `a` and `e` shows us the relationship between the average distance from the Sun and the orbital shape. The visualization does not show a relationship between `a` and `e` because there is not a `linear` relationship. The visualization does show that small-bodies tend to hold specific orbital distances with a major segment between 2 and 3.5 AU and another smaller segment around 5.2 AU. This is consistent with the main asteroid belt between Mars and Jupiter and a smaller one at Jupiter's orbit.

In [23]:
```
sns.set_theme(style="whitegrid")
plt.figure(figsize=(10, 6))

scatter = plt.scatter(df_clean['a'], df_clean['e'], alpha=0.5, s=8)
plt.title('Average Distance from Sun vs. Oval Shape', fontsize=14, pad=15)
plt.xlabel('a (Semi-Major Axis in AU)', fontsize=12)
plt.ylabel('e (Eccentricity)', fontsize=12)
plt.xlim(0, 10)
plt.tight_layout()
```

```
plt.savefig("figures/eccentricity_v_semi_major_axis.png", dpi=300)
plt.show()
```



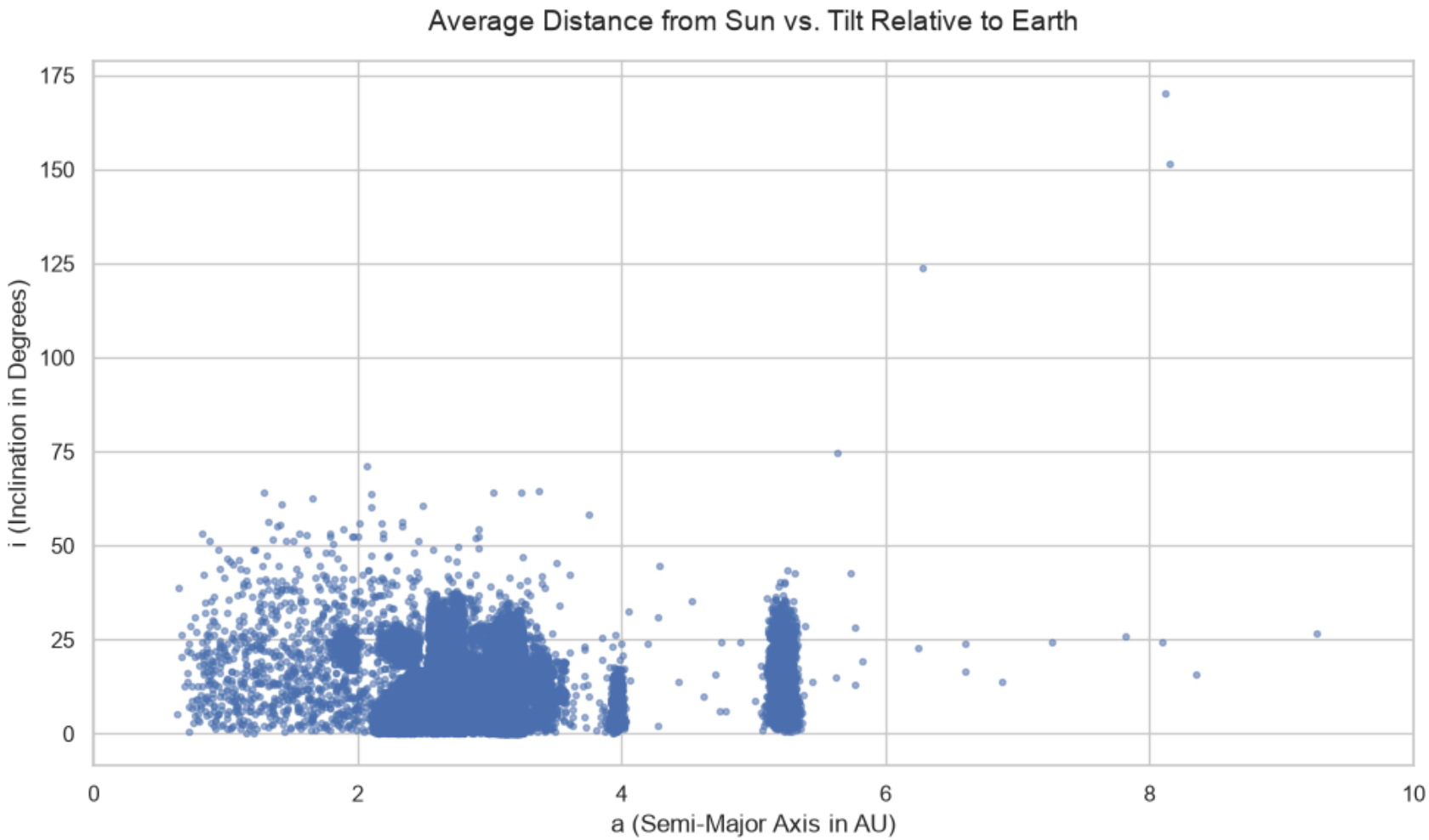

### Semi-Major Axis v. Inclination

Visualizing `a` and `i` shows us the relationship between the average distance from the Sun and the inclination relative to Earth. The visualization does not show a relationship between `a` and `i` because there is not a linear relationship. The visualization does show that small-bodies throughout the inner solar system tend to have the same inclination as Earth.

In [24]:
```
sns.set_theme(style="whitegrid")
plt.figure(figsize=(10, 6))

scatter = plt.scatter(df_clean['a'], df_clean['i'], alpha=0.5, s=8)
plt.title('Average Distance from Sun vs. Tilt Relative to Earth', fontsize=14, pad=15)
plt.xlabel('a (Semi-Major Axis in AU)', fontsize=12)
plt.ylabel('i (Inclination in Degrees)', fontsize=12)
plt.xlim(0, 10)
plt.tight_layout()

plt.savefig("figures/inclination_v_semi_major_axis.png", dpi=300)
plt.show()
```




### Eccentricity v. Inclination

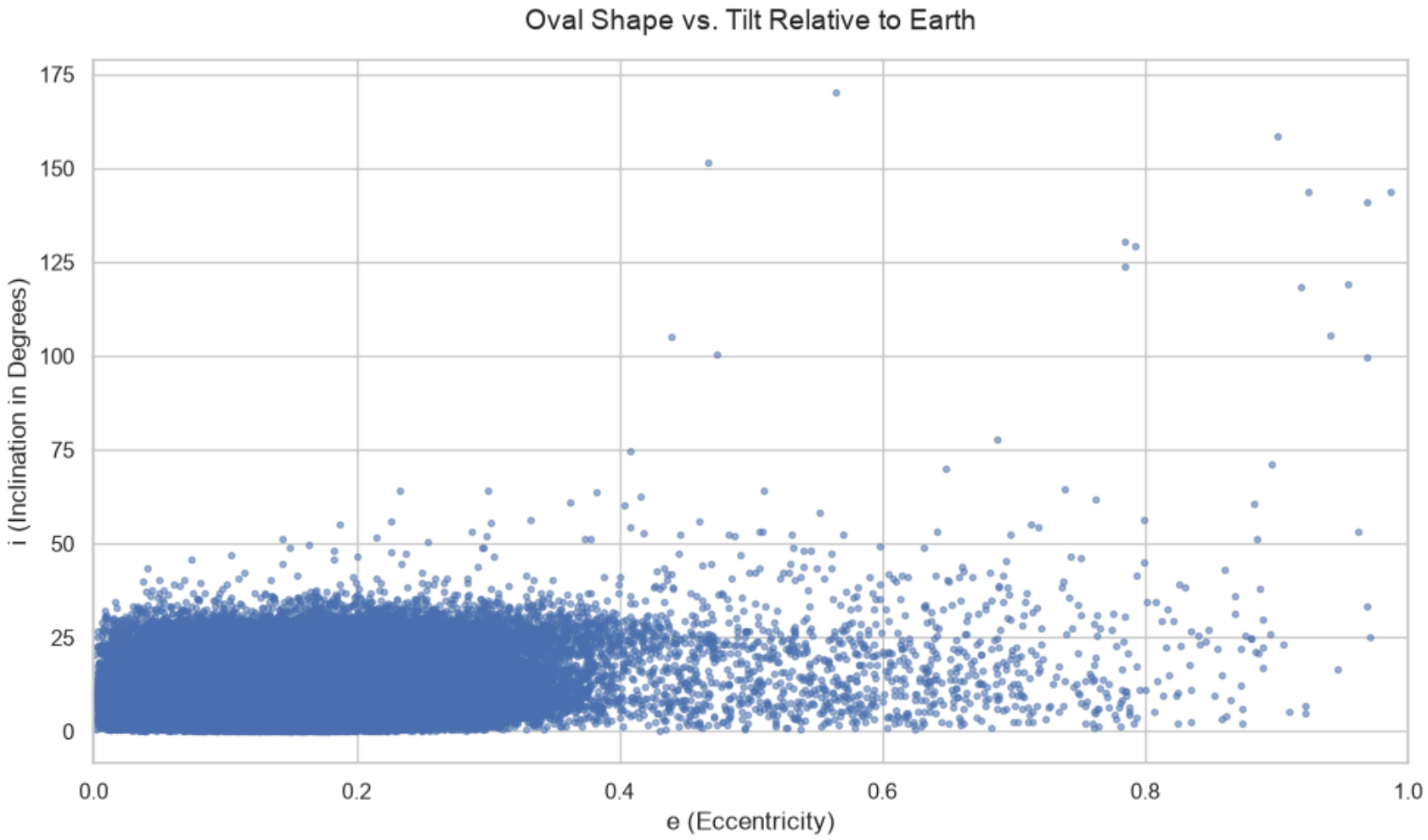
Visualizing `e` and `i` shows us the relationship between the orbital shape and the elliptical plane. The visualization does not show a correlation between `e` and `i` because there is not a linear relationship.

In [25]:
```
sns.set_theme(style="whitegrid")
plt.figure(figsize=(10, 6))

scatter = plt.scatter(df_clean['e'], df_clean['i'], alpha=0.5, s=8)
plt.title('Oval Shape vs. Tilt Relative to Earth', fontsize=14, pad=15)
```

```
plt.xlabel('e (Eccentricity)', fontsize=12)
plt.ylabel('i (Inclination in Degrees)', fontsize=12)
plt.xlim(0, 1)
plt.tight_layout()

plt.savefig("figures/eccentricity_v_inclination.png", dpi=300)
plt.show()
```




## 5. Excluded Features

`om` and `w` were excluded from visualization as the feature correlation matrix determined there was not a strong correlation between these features and other features or the `target`. While these features may or may not introduce noise to the tree-based regression models, peer-reviewed analysis has determined that they do not provide strong predictive indicators for `diameter`. Due to the computing power and time to analyze additional features in the dataset, these features will only analyzed as an ad hoc if time permits prior to the project deadline.

In [26]:
```
df_clean = df_clean.drop(columns=['om', 'w'])
df_clean.head()
```

Out[26]:

| | diameter | a | e | i | H | q | diameter_log |
|---|---|---|---|---|---|---|---|
| **0** | 939.400 | 2.766 | 0.0797 | 10.59 | 3.34 | 2.545 | 2.972851 |
| **1** | 513.000 | 2.770 | 0.2307 | 34.93 | 4.12 | 2.131 | 2.710117 |
| **2** | 246.596 | 2.671 | 0.2557 | 12.99 | 5.19 | 1.988 | 2.391986 |
| **3** | 522.770 | 2.361 | 0.0902 | 7.14 | 3.25 | 2.148 | 2.718311 |
| **4** | 106.699 | 2.577 | 0.1876 | 5.36 | 6.95 | 2.093 | 2.028160 |

In [27]:
```
# save cleaned data to csv
df_clean.to_csv(dataset_cleaned_path, index=False)
```

## 1. Exploratory Data Analysis (EDA)

Preliminary analysis of the dataset shows that the key indicators of a small-body size (`target` `diameter` (`Y`)) are `absolute magnitude` (`H`), `semi-major axis` (`a`), `eccentricity` (`e`), and `inclination` (`i`). There is a strong `non-linear` relationship between `diameter` and `H` indicating that `H` is a strong predictor of `diameter`. While there were no `linear` correlations between individual `a`, `e`, `i` features and `diameter` or `H`, comparison of `a`, `e`, and `i` mapped to `diameter` and `a`, `e`, and `i` mapped to `H` provided similar results. This provides strong evidence that `a`, `e`, and `i` could meet the operational threshold to predict `diameter` without the need for `H`. The `EDA` alone is not enough to conclude if the `null hypothesis` or the `alternative hypothesis` are correct, but there is a pattern and relationship between `a`, `e`, `i`, and `diameter`.

## 2. Feature Determination

Analysis of the features indicates that `a`, `e`, and `i` are strong candidates for the regression model because they have a `non-linear`, `structural` relationship to the target. The feature `q` is reserved for future determination for inclusion or exclusion in the regression model since it is derived from `a` and `e` which could introduce `multicollinearity` or `noise`. Features `om` and `w` can be excluded. While `om` and `w` are not derived from any other features, the lack of correlation to the `target` may introduce `noise` to the regression model that could lead to `overfitting`.

### 3. Model Selection

An analysis of the dataset and feature determination support using a `random forest` regression model to test the hypothesis. Small-bodies form dense structural clustering in the feature space and a highly skewed `target` distribution. Small-bodies also can be observed at great distances which creates mathematical `outliers`. Physical characteristics and derived characteristics may have no correlation to the size of a small-body or introduce `multicollinearity`. A `random forest` regression model uses piecewise step-function approximations to handle extreme `outliers` and `noise` of derived or irrelevant features. This is better than `linear` regression or other regression models that try to fit data into a straight line or curve. This will result in stronger models to increase accuracy to predict small-body sizes and to test the hypothesis.

## III. ANALYZE (MODELS)

Ran six regression models on the cleaned dataset. These models were chosen because they can best handle the non-linear, heavily skewed data of `asteroids`. The two classical machine learning models are `Random Forest` (`RF`) and `Gradient Boosting Regression` (`GBR`, `XGBoost` in code) which utilize decision-trees to ask questions to split data into distinct numerical groups to calculate a continuous prediction value.

The four deep learning models are `Wide & Deep Network` (`W&D`), `Tabular ResNet` (`TabResNet`), `FT-Transformer` and `Gated ResNet` (`GRN`) used because they handle tabular data like in the structured dataset. `W&D` is used because the wide path can learn the known mathematical rules like the correlation of `absolute magnitude` (`H`) and `target` `diameter` while the deep path can learn any subtle patterns with the orbital characteristics (`a`, `e`, `i`). `TabResNet` minimizes the effects of vanishing gradient problem as the network gets deeper. `FT-Transformer` was selected to map the complex dependencies of the features together. It looks at them as a unified system rather than isolated variables. `GRN` was selected because it is similar to `TabResNet` but will use gates to filter out noisy data to help predict the patterns in orbital characteristics, specifically in the restricted subset.

Each model was trained with the full and restricted subsets and the results logged for analysis. Training utilized a five-fold cross validation where each fold and fold results were logged as well.

In [28]:
```
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import pprint
from sklearn.metrics import r2_score
import plotly.graph_objects as go
import ipywidgets as widgets
from ipywidgets import interact
from sklearn.feature_selection import mutual_info_regression
from asteroid import train_model, predict_model, load_config
import visualization_utils as vutils

TEST_RUN = False
```

## Model Training

The models were trained and tested against the hypotheses, quantified as:

$$\Delta\ R^2_{test} = R^2_{Model(\mathbf{X}_{full})} - R^2_{Model(\mathbf{X}_{restricted})}$$

$\Delta\ R^2_{test}$ was used to reject or fail to reject the null hypotheses. Every model resulted in $\Delta\ R^2_{test}$ greater than the statistical and operational threshold ($\delta = 0.05$). This means that the restricted features subset was not sufficient to accurately predict the `diameter` of an `asteroid` for operational use. As such, final model determination was based on the greatest full feature subset model mean $R^2$. While all models met or exceeded a mean $R^2$ of `0.7887`, it was the traditional models of `Random Forest` and `Gradient Boosting Regression` that scored the best with a mean $R^2$ of `0.8508` and `0.8548`, respectively. The determination of which model will be used in production will undergo an analysis of average `Mean Absolute Percentage Error` (`MAPE`) to determine how well predictions matched actual values of `asteroids`. The model with the lowest average `MAPE` will be selected for use in production.

In [ ]:
```
experiments = [
    ("rf", None, "Random Forest Base"),
    ("xgboost", None, "XGBoost Base"),
    ("keras_wide_deep", {"wide_dim": 1}, "Keras Wide & Deep Neural Network"),
    ("keras_resnet", {"batch_size": 512, "hidden_dim": 256, "num_blocks": 3, "learning_rate": 0.0005}, "Ke
    ("keras_ft_transformer", {"batch_size": 512, "embedding_dim": 16, "num_heads": 2, "dropout_rate": 0.05
    ("keras_gated_resnet", {"batch_size": 512, "hidden_dim": 256, "num_blocks": 3, "learning_rate": 0.0005
]

for i, (engine, overrides, label) in enumerate(experiments, 1):
    mode_label = "Testing" if TEST_RUN else "Training"
    print(f"\n--- [Run {i}/{len(experiments)}] {mode_label} {label} ---")
    active_overrides = overrides.copy() if overrides else {}

    if TEST_RUN:
        if engine.startswith("keras_") or engine in ["wide_deep", "resnet", "gated_resnet", "ft_transforme
            active_overrides.update({"epochs": 1, "patience": 1, "batch_size": 64})
        elif engine in ["rf", "xgboost", "xgb", "gradient"]:
            active_overrides.update({"n_estimators": 2, "max_depth": 2})
    train_model(engine_type=engine, override_params=active_overrides, delta_threshold=0.05, test_mode=TEST_

print(f"\n{len(experiments)} experiment run(s) completed.")
```

*Results located in logs/production_run_output.txt*

**INITIALIZING RANDOM FOREST PIPELINE**

Dataset Path: data/cleaned-dataset-jpl-small-body-asteroids.csv

Dataset Shape: (139203, 7)

Full Features: ['a', 'e', 'i', 'H']

Restrictive Features: ['a', 'e', 'i']

Target Column: diameter_log

Model Parameters: {'n_estimators': 150, 'max_depth': 25, 'min_samples_split': 5, 'random_state': 42, 'n_jobs': -1}

| **Fold 1/5 Complete** | **Full R²: 0.8492** | **Restricted R²: 0.3319** | **Δ R²: 0.5173 ** |
|---|---|---|---|
| Fold 3/5 Complete | Full R²: 0.8513 | Restricted R²: 0.3325 | Δ R²: 0.5188 \ |
| Fold 4/5 Complete | Full R²: 0.8473 | Restricted R²: 0.3321 | Δ R²: 0.5152 \ |

| Fold 1/5 Complete | Full $R^2$: 0.8492 | Restricted $R^2$: 0.3319 | Δ $R^2$: 0.5173 \ |
|---|---|---|---|
| Fold 5/5 Complete | Full $R^2$: 0.8549 | Restricted $R^2$: 0.3327 | Δ $R^2$: 0.5222 |

**CROSS-VALIDATION SUMMARY AVERAGES (RANDOM FOREST)**

Full Model Mean $R^2$: 0.8508

Restricted Model Mean $R^2$: 0.3330

Mean Δ $R^2$ (Drop Impact): 0.5178 (± 0.0026)

Full Model Mean MAE: 1.0443 km

Restricted Model Mean MAE:2.4090 km

**HYPOTHESIS TESTING REPORT (RANDOM FOREST)**

Testing against Operational Tolerance (δ) = 0.05

Calculated Delta $R^2$ (Drop Impact): = 0.5178

Statistical Test: [H1_stat] -> Reject H0_stat: Group 2 features significantly improve out-of-sample $R^2$.

Operational Test: [H0_ops] -> Fail to Reject H0_ops: Group 2 feature improvement is > delta (0.05). Restricted subset is insufficient.

---

**INITIALIZING GBR PIPELINE**

Dataset Path: data/cleaned-dataset-jpl-small-body-asteroids.csv

Dataset Shape: (139203, 7)

Full Features: ['a', 'e', 'i', 'H']

Restrictive Features: ['a', 'e', 'i']

Target Column: diameter_log

Model Parameters: {'n_estimators': 150, 'max_depth': 8, 'learning_rate': 0.05, 'min_samples_leaf': 2, 'random_state': 42}

| Fold 1/5 Complete | Full $R^2$: 0.8527 | Restricted $R^2$: 0.3582 | Δ $R^2$: 0.4945 \ |
|---|---|---|---|
| Fold 3/5 Complete | Full $R^2$: 0.8557 | Restricted $R^2$: 0.3659 | Δ $R^2$: 0.4898 \ |
| Fold 4/5 Complete | Full $R^2$: 0.8524 | Restricted $R^2$: 0.3652 | Δ $R^2$: 0.4872 \ |
| Fold 5/5 Complete | Full $R^2$: 0.8584 | Restricted $R^2$: 0.3653 | Δ $R^2$: 0.4931 |

**CROSS-VALIDATION SUMMARY AVERAGES (GBR)**

Full Model Mean $R^2$: 0.8548

Restricted Model Mean $R^2$: 0.3630

Mean Δ $R^2$ (Drop Impact): 0.4917 (± 0.0028)

Full Model Mean MAE: 1.0360 km

Restricted Model Mean MAE:2.3461 km

**HYPOTHESIS TESTING REPORT (GBR)**

Testing against Operational Tolerance (δ) = 0.05

Calculated Delta $R^2$ (Drop Impact): = 0.4917

Statistical Test: [H1_stat] -> Reject H0_stat: Group 2 features significantly improve out-of-sample $R^2$.

Operational Test: [H0_ops] -> Fail to Reject H0_ops: Group 2 feature improvement is > delta (0.05). Restricted subset is insufficient.

---

**INITIALIZING KERAS WIDE_DEEP NETWORK PIPELINE**

Dataset Path: data/cleaned-dataset-jpl-small-body-asteroids.csv

Dataset Shape: (139203, 10)

Full Features: ['a', 'e', 'i', 'H']

Restrictive Features: ['a', 'e', 'i']

Target Column: diameter_log

Model Parameters: {'epochs': 100, 'batch_size': 64, 'learning_rate': 0.001, 'patience': 10, 'restore_best_weights': True, 'wide_dim': 1}

| **Fold 1/5 Complete** | **Full $R^2$: 0.8006** | **Restricted $R^2$: 0.2562** | **Δ $R^2$: 0.5444 ** |
|---|---|---|---|
| Fold 3/5 Complete | Full $R^2$: 0.8309 | Restricted $R^2$: 0.3083 | Δ $R^2$: 0.5226 \ |
| Fold 4/5 Complete | Full $R^2$: 0.7944 | Restricted $R^2$: 0.2230 | Δ $R^2$: 0.5715 \ |
| Fold 5/5 Complete | Full $R^2$: 0.7732 | Restricted $R^2$: 0.2978 | Δ $R^2$: 0.4755 |

**CROSS-VALIDATION SUMMARY AVERAGES (WIDE_DEEP)**

Full Model Mean $R^2$: 0.8053

Restricted Model Mean $R^2$: 0.2090

Mean Δ $R^2$ (Drop Impact): 0.5963 (± 0.1392)
Full Model Mean MAE: 1.2608 km
Restricted Model Mean MAE:4098258727372552614707 2.0000 km

**HYPOTHESIS TESTING REPORT (WIDE_DEEP)**
Testing against Operational Tolerance (δ) = 0.0500
Calculated Delta $R^2$ (Drop Impact): = 0.5963
Statistical Test: [H1_stat] -> Reject H0_stat: Group 2 features significantly improve out-of-sample $R^2$.
Operational Test: [H0_ops] -> Fail to Reject H0_ops: Group 2 feature improvement is > delta (0.05). Restricted subset is insufficient.

---

**INITIALIZING KERAS RESNET NETWORK PIPELINE**
Dataset Path: data/cleaned-dataset-jpl-small-body-asteroids.csv
Dataset Shape: (139203, 10)
Full Features: ['a', 'e', 'i', 'H']
Restrictive Features: ['a', 'e', 'i']
Target Column: diameter_log
Model Parameters: {'epochs': 100, 'batch_size': 512, 'learning_rate': 0.0005, 'patience': 10, 'restore_best_weights': True, 'hidden_dim': 256, 'num_blocks': 3}

| **Fold 1/5 Complete** | **Full $R^2$: 0.8166** | **Restricted $R^2$: 0.0875** | **Δ $R^2$: 0.7290** |
|---|---|---|---|
| Fold 3/5 Complete | Full $R^2$: 0.8093 | Restricted $R^2$: 0.2519 | Δ $R^2$: 0.5574 |
| Fold 4/5 Complete | Full $R^2$: 0.8103 | Restricted $R^2$: 0.3016 | Δ $R^2$: 0.5087 |
| Fold 5/5 Complete | Full $R^2$: 0.8409 | Restricted $R^2$: 0.3039 | Δ $R^2$: 0.5370 |

**CROSS-VALIDATION SUMMARY AVERAGES (RESNET)**
Full Model Mean $R^2$: 0.8189 Restricted Model Mean $R^2$: 0.2487 Mean Δ $R^2$ (Drop Impact): 0.5702 (± 0.0811) Full Model Mean MAE: 1.2734 km Restricted Model Mean MAE:8.3245 km

**HYPOTHESIS TESTING REPORT (RESNET)**
Testing against Operational Tolerance (δ) = 0.0500
Calculated Delta $R^2$ (Drop Impact): = 0.5702

Statistical Test: [H1_stat] -> Reject H0_stat: Group 2 features significantly improve out-of-sample R².
Operational Test: [H0_ops] -> Fail to Reject H0_ops: Group 2 feature improvement is > delta (0.05). Restricted subset is insufficient.

---

**INITIALIZING KERAS FT_TRANSFORMER NETWORK PIPELINE**

Dataset Path: data/cleaned-dataset-jpl-small-body-asteroids.csv
Dataset Shape: (139203, 10)
Full Features: ['a', 'e', 'i', 'H']
Restrictive Features: ['a', 'e', 'i']
Target Column: diameter_log
Model Parameters: {'epochs': 100, 'batch_size': 512, 'learning_rate': 0.0003, 'patience': 10, 'restore_best_weights': True, 'embedding_dim': 16, 'num_heads': 2, 'dropout_rate': 0.05}

| **Fold 1/5 Complete** | **Full R²: 0.8290** | **Restricted R²: 0.3032** | **Δ R²: 0.5258 ** |
|---|---|---|---|
| Fold 3/5 Complete | Full R²: 0.8223 | Restricted R²: 0.3081 | Δ R²: 0.5143 \ |
| Fold 4/5 Complete | Full R²: 0.8223 | Restricted R²: 0.3161 | Δ R²: 0.5061 \ |
| Fold 5/5 Complete | Full R²: 0.8358 | Restricted R²: 0.3157 | Δ R²: 0.5202 \ |

**CROSS-VALIDATION SUMMARY AVERAGES (FT_TRANSFORMER)**

Full Model Mean R²: 0.8280
Restricted Model Mean R²: 0.3104
Mean Δ R² (Drop Impact): 0.5177 (± 0.0069)
Full Model Mean MAE: 1.1435 km
Restricted Model Mean MAE:2.4215 km \

**HYPOTHESIS TESTING REPORT (FT_TRANSFORMER)**

Testing against Operational Tolerance (δ) = 0.0500
Calculated Delta R² (Drop Impact): = 0.5177
Statistical Test: [H1_stat] -> Reject H0_stat: Group 2 features significantly improve out-of-sample R².
Operational Test: [H0_ops] -> Fail to Reject H0_ops: Group 2 feature improvement is > delta (0.05). Restricted subset is insufficient.

---

**INITIALIZING KERAS GATED_RESNET NETWORK PIPELINE**

Dataset Path: data/cleaned-dataset-jpl-small-body-asteroids.csv

Dataset Shape: (139203, 10)

Full Features: ['a', 'e', 'i', 'H']

Restrictive Features: ['a', 'e', 'i']

Target Column: diameter_log

Model Parameters: {'epochs': 100, 'batch_size': 512, 'learning_rate': 0.0005, 'patience': 10, 'restore_best_weights': True, 'hidden_dim': 256, 'num_blocks': 3}

| **Fold 1/5 Complete** | **Full R²: 0.8229** | **Restricted R²: 0.1872** | **Δ R²: 0.6358 \|** |
|---|---|---|---|
| Fold 3/5 Complete | Full R²: 0.7274 | Restricted R²: 0.0450 | Δ R²: 0.6824 \| |
| Fold 4/5 Complete | Full R²: 0.7397 | Restricted R²: -0.0532 | Δ R²: 0.7929 \| |
| Fold 5/5 Complete | Full R²: 0.8351 | Restricted R²: 0.2660 | Δ R²: 0.5691 |

**CROSS-VALIDATION SUMMARY AVERAGES (GATED_RESNET)**

Full Model Mean R²: 0.7887

Restricted Model Mean R²: 0.1492

Mean Δ R² (Drop Impact): 0.6396 (± 0.0950)

Full Model Mean MAE: 4.4961 km

Restricted Model Mean MAE:25549384556834445066240.0000 km

**HYPOTHESIS TESTING REPORT (GATED_RESNET)**

Testing against Operational Tolerance (δ) = 0.0500

Calculated Delta R² (Drop Impact): = 0.6396

Statistical Test: [H1_stat] -> Reject H0_stat: Group 2 features significantly improve out-of-sample R². Operational Test: [H0_ops] -> Fail to Reject H0_ops: Group 2 feature improvement is > delta (0.05). Restricted subset is insufficient.

A subset of asteroids diameters were predicted with each model (full and restricted trained models) ranging from `Sub-Kilometer`, `Medium`, and `Massive` to validate accuracy and the results logged for analysis.

In [29]:
```
asteroids = [
    {"spkid": 20000001, "name": "Ceres", "actual": 939.4, "data": {"a": 2.766, "e": 0.0797, "i": 10.59, "H"
    {"spkid": 20020000, "name": "Varuna", "actual": 900.0, "data": {"a": 43.2, "e": 0.0516, "i": 17.14, "H"
```

```
    {"spkid": 20000004, "name": "Vesta", "actual": 522.77, "data": {"a": 2.361, "e": 0.0902, "i": 7.14, "H"
    {"spkid": 20000002, "name": "Pallas", "actual": 513.0, "data": {"a": 2.77, "e": 0.2307, "i": 34.93, "H"
    {"spkid": 20000003, "name": "Juno", "actual": 246.596, "data": {"a": 2.671, "e": 0.2557, "i": 12.99, "
    {"spkid": 20000065, "name": "Cybele", "actual": 237.26, "data": {"a": 3.407, "e": 0.1279, "i": 3.57, "
    {"spkid": 20000560, "name": "Delila", "actual": 35.334, "data": {"a": 2.753, "e": 0.1579, "i": 8.45, "
    {"spkid": 20001203, "name": "Nanna", "actual": 35.18, "data": {"a": 2.882, "e": 0.2515, "i": 6.0, "H":
    {"spkid": 20001815, "name": "Beethoven", "actual": 30.598, "data": {"a": 3.149, "e": 0.1932, "i": 2.74
    {"spkid": 20004864, "name": "Nimoy", "actual": 11.874, "data": {"a": 2.468, "e": 0.1782, "i": 3.62, "H"
    {"spkid": 20012358, "name": "Azzurra", "actual": 11.75, "data": {"a": 2.79, "e": 0.1143, "i": 7.61, "H"
    {"spkid": 20013493, "name": "Lockwood", "actual": 11.101, "data": {"a": 2.668, "e": 0.1995, "i": 12.91
    {"spkid": 20005855, "name": "Yukitsuna", "actual": 11.1, "data": {"a": 2.555, "e": 0.154, "i": 15.5, "
    {"spkid": 20013123, "name": "Tyson", "actual": 10.875, "data": {"a": 2.359, "e": 0.2725, "i": 23.27, "
    {"spkid": 20001946, "name": "Walraven", "actual": 9.205, "data": {"a": 2.294, "e": 0.2345, "i": 8.16, "
    {"spkid": 20000970, "name": "Primula", "actual": 9.204, "data": {"a": 2.559, "e": 0.2723, "i": 5.03, "
    {"spkid": 20002445, "name": "Blazhko", "actual": 9.13, "data": {"a": 2.269, "e": 0.1467, "i": 6.07, "H"
    {"spkid": 20008626, "name": "Melissarauch", "actual": 4.266, "data": {"a": 3.169, "e": 0.1038, "i": 6.1
    {"spkid": 20012503, "name": "Danielpeterson", "actual": 4.266, "data": {"a": 2.756, "e": 0.1282, "i": 8
    {"spkid": 20003072, "name": "Vilnius", "actual": 4.26, "data": {"a": 2.239, "e": 0.1799, "i": 5.65, "H"
    {"spkid": 20091287, "name": "Simon-Garfunkel", "actual": 1.745, "data": {"a": 2.289, "e": 0.1076, "i":
    {"spkid": 20001865, "name": "Cerberus", "actual": 1.2, "data": {"a": 1.08, "e": 0.4669, "i": 16.1, "H"
    {"spkid": 20001915, "name": "Quetzalcoatl", "actual": 0.5, "data": {"a": 2.545, "e": 0.5703, "i": 20.4
    {"spkid": 20003757, "name": "Anagolay", "actual": 0.5, "data": {"a": 1.835, "e": 0.4458, "i": 3.87, "H"
    {"spkid": 20101955, "name": "Bennu", "actual": 0.48444, "data": {"a": 1.126, "e": 0.2037, "i": 6.03, "
    {"spkid": 20398188, "name": "Agni", "actual": 0.462, "data": {"a": 0.8646, "e": 0.2734, "i": 13.25, "H"
    {"spkid": 20004660, "name": "Nereus", "actual": 0.33, "data": {"a": 1.485, "e": 0.3587, "i": 1.45, "H"
]

results_records = []
for ast in asteroids:
    spkid_id, name_ast, actual_dia, sample_data = ast["spkid"], ast["name"], ast["actual"], ast["data"].co

    sample_data["spkid"] = spkid_id
    sample_data["Asteroid"] = name_ast
    sample_data["Actual Diameter (km)"] = actual_dia

    engines_list = [
        ("rf", "Random Forest"),
        ("xgboost", "XGBoost"),
        ("keras_wide_deep", "Keras Wide & Deep"),
        ("keras_resnet", "Keras Tabular ResNet"),
        ("keras_ft_transformer", "Keras FT-Transformer"),
        ("keras_gated_resnet", "Keras Gated ResNet")
```

```
    ]

    for engine, engine_name in engines_list:
        preds = predict_model(sample_data, engine_type=engine, model_type="both", test_mode=TEST_RUN)
        full_p, rest_p = preds['full_model_prediction'], preds['restricted_model_prediction']
        full_err_pct = (abs(full_p - actual_dia) / actual_dia) * 100
        rest_err_pct = (abs(rest_p - actual_dia) / actual_dia) * 100

        results_records.append({
            "spkid": spkid_id, "Asteroid": name_ast, "Actual Diameter (km)": actual_dia, "Engine": engine_
            "Full Pred (km)": round(full_p, 2), "Full Error %": round(full_err_pct, 1),
            "Restricted Pred (km)": round(rest_p, 2), "Restricted Error %": round(rest_err_pct, 1)
        })

df_summary = pd.DataFrame(results_records)
print("\nOVERALL PREDICTION SUMMARY TABLE:\n" + "=" * 95)
display(df_summary)
print("=" * 95 + "\n\nMean Absolute Percentage Error (MAPE) by Engine for Full Model:")
print(df_summary.groupby("Engine")["Full Error %"].mean().map("{:.2f}%".format))
```

```
OVERALL PREDICTION SUMMARY TABLE:
===============================================================================================
```

| | spkid | Asteroid | Actual Diameter (km) | Engine | Full Pred (km) | Full Error % | Restricted Pred (km) | Restricted Error % |
|---|---|---|---|---|---|---|---|---|
| **0** | 20000001 | Ceres | 939.40 | Random Forest | 633.16 | 32.6 | 42.79 | 95.4 |
| **1** | 20000001 | Ceres | 939.40 | XGBoost | 783.44 | 16.6 | 4.29 | 99.5 |
| **2** | 20000001 | Ceres | 939.40 | Keras Wide & Deep | 885.13 | 5.8 | 4.34 | 99.5 |
| **3** | 20000001 | Ceres | 939.40 | Keras Tabular ResNet | 701.61 | 25.3 | 4.59 | 99.5 |
| **4** | 20000001 | Ceres | 939.40 | Keras FT-Transformer | 250.44 | 73.3 | 4.08 | 99.6 |
| **...** | ... | ... | ... | ... | ... | ... | ... | ... |
| **157** | 20004660 | Nereus | 0.33 | XGBoost | 0.48 | 45.2 | 0.19 | 42.9 |
| **158** | 20004660 | Nereus | 0.33 | Keras Wide & Deep | 0.49 | 48.0 | 0.32 | 1.5 |
| **159** | 20004660 | Nereus | 0.33 | Keras Tabular ResNet | 0.73 | 122.5 | 0.26 | 20.6 |
| **160** | 20004660 | Nereus | 0.33 | Keras FT-Transformer | 0.68 | 104.8 | 0.25 | 25.5 |
| **161** | 20004660 | Nereus | 0.33 | Keras Gated ResNet | 0.83 | 151.6 | 1.18 | 256.2 |

162 rows × 8 columns

```
==========================================================================================

Mean Absolute Percentage Error (MAPE) by Engine for Full Model:
Engine
Keras FT-Transformer     34.42%
Keras Gated ResNet      105.87%
Keras Tabular ResNet     32.39%
Keras Wide & Deep        37.36%
Random Forest            14.43%
XGBoost                  20.79%
Name: Full Error %, dtype: str
```

The `Mean Absolute Percentage Error` (`MAPE`) and volatility (`standard deviation`) matrix was used to see how well each model predicted the size of an asteroid. It was broken down into categories by `asteroid` sizes (`Massive`, `Medium`, and `Sub-Kilometer`) to see how well it managed the skewed data. The results showed that the classical models had the best average among all categories with `Random Forest` having a `MAPE` of `14.44%` and `GBR` (as `xgboost` in code) having a `20.79%` for the full feature model. This means the top classical models deviate from the actual `diameter` by an average of only `15%` to `21%` across all samples.

In general, each model struggled with the skewed data specifically with the `Sub-Kilometer` categories. The `MAPE` score for all models in this category skyrocked from the `RF` at `20.37%` up to `GRN` at `83.41%`. This makes sense as there are many more smaller `asteroids` and smaller `asteroids` tend to be harder to gather data on.

Most notable is the complete collapse of all accuracy in predicting `diameter` of an `asteroid` in the restricted feature subset. `MAPE` for all models skyrockted from an average of `49.47%` with `RF` up to `99.06%` with `GRN`. This suggests that `absolute magnitude` is highly correlated to `diameter` and orbital characterisitcs alone can not reliably predicit the size of an asteroid without `H`.

In [30]:
```
df_logs = pd.read_csv('logs/prediction_results.csv')

conditions = [
    (df_logs['Actual Diameter (km)'] < 1.0),
    (df_logs['Actual Diameter (km)'].between(1.0, 50.0)),
    (df_logs['Actual Diameter (km)'] > 50.0)
]
choices = ['Sub-Kilometer (<1km)', 'Medium (1-50km)', 'Massive (>50km)']
df_logs['Size_Bucket'] = np.select(conditions, choices, default='Unknown')
df_logs = df_logs.sort_index(kind='stable')
if 'Engine' in df_logs.columns and 'Subset' in df_logs.columns:
    df_logs = df_logs.sort_values(by=['Engine', 'Subset', 'Size_Bucket'], kind='stable').reset_index(drop='

column_order = [
    'Full: Avg MAPE', 'Full: Error Volatility (Std)',
    'Restricted: Avg MAPE', 'Restricted: Error Volatility (Std)'
]

matrix_blocks = []
unique_engines = sorted(df_logs['Engine'].unique())
```

```
for eng in unique_engines:
    df_eng = df_logs[df_logs['Engine'] == eng]

    df_variance = df_eng.groupby(['Engine', 'Subset', 'Size_Bucket'])['Error %'].agg(
        Avg_MAPE='mean',
        Std_Volatility='std'
    )

    df_pivoted = df_variance.unstack(level='Subset')
    df_pivoted.columns = [
        'Full: Avg MAPE', 'Restricted: Avg MAPE',
        'Full: Error Volatility (Std)', 'Restricted: Error Volatility (Std)'
    ]
    df_pivoted = df_pivoted[column_order]
    matrix_blocks.append(df_pivoted)

    df_sub_raw = df_eng.groupby(['Subset'])['Error %'].agg(
        Avg_MAPE='mean',
        Std_Volatility='std'
    )

    sub_data = {
        'Full: Avg MAPE': [df_sub_raw.loc['full', 'Avg_MAPE'] if 'full' in df_sub_raw.index else np.nan],
        'Full: Error Volatility (Std)': [df_sub_raw.loc['full', 'Std_Volatility'] if 'full' in df_sub_raw.
        'Restricted: Avg MAPE': [df_sub_raw.loc['restricted', 'Avg_MAPE'] if 'restricted' in df_sub_raw.in
        'Restricted: Error Volatility (Std)': [df_sub_raw.loc['restricted', 'Std_Volatility'] if 'restricte
    }

    df_sub_avg = pd.DataFrame(sub_data, index=pd.MultiIndex.from_tuples([(eng, 'Average')]))
    df_sub_avg = df_sub_avg[column_order]
    matrix_blocks.append(df_sub_avg)

df_final_matrix = pd.concat(matrix_blocks)

print("\n" + "="*95 + "\nMAPE & VOLATILITY MATRIX WITH MODEL\n" + "="*95)
display(df_final_matrix.style.format("{:.2f}%", na_rep="N/A"))
```

```
===============================================================================================
MAPE & VOLATILITY MATRIX WITH MODEL
===============================================================================================
```

| | | Full: Avg MAPE | Full: Error Volatility (Std) | Restricted: Avg MAPE | Restricted: Error Volatility (Std) |
|---|---|---|---|---|---|
| keras_ft_transformer | Massive (>50km) | 52.24% | 23.14% | 98.51% | 1.17% |
| | Medium (1-50km) | 24.72% | 17.86% | 63.60% | 21.15% |
| | Sub-Kilometer (<1km) | 44.13% | 47.00% | 60.16% | 50.76% |
| | Average | 34.43% | 27.86% | 70.72% | 29.76% |
| keras_gated_resnet | Massive (>50km) | 326.62% | 700.98% | 98.86% | 0.86% |
| | Medium (1-50km) | 30.10% | 18.53% | 59.85% | 25.12% |
| | Sub-Kilometer (<1km) | 83.41% | 77.87% | 224.78% | 68.57% |
| | Average | 105.87% | 332.42% | 99.06% | 71.22% |
| keras_resnet | Massive (>50km) | 19.07% | 7.24% | 91.34% | 18.58% |
| | Medium (1-50km) | 25.81% | 18.43% | 64.81% | 22.69% |
| | Sub-Kilometer (<1km) | 69.42% | 64.14% | 59.46% | 73.57% |
| | Average | 32.39% | 34.21% | 69.71% | 36.59% |
| keras_wide_deep | Massive (>50km) | 21.52% | 20.06% | 83.56% | 37.26% |
| | Medium (1-50km) | 40.31% | 26.15% | 62.89% | 22.91% |
| | Sub-Kilometer (<1km) | 46.88% | 40.10% | 82.55% | 109.68% |
| | Average | 37.35% | 28.28% | 71.13% | 50.23% |
| rf | Massive (>50km) | 18.03% | 12.48% | 86.25% | 12.45% |
| | Medium (1-50km) | 11.24% | 9.32% | 41.49% | 19.02% |
| | Sub-Kilometer (<1km) | 20.37% | 20.39% | 30.84% | 21.48% |
| | Average | 14.44% | 12.65% | 49.47% | 26.97% |

| | | Full: Avg MAPE | Full: Error Volatility (Std) | Restricted: Avg MAPE | Restricted: Error Volatility (Std) |
|---|---|---|---|---|---|
| xgboost | Massive (>50km) | 8.10% | 4.20% | 85.92% | 24.31% |
| | Medium (1-50km) | 20.81% | 17.78% | 59.50% | 22.07% |
| | Sub-Kilometer (<1km) | 35.98% | 30.47% | 27.04% | 22.17% |
| | Average | 20.80% | 20.25% | 59.36% | 28.88% |

Looking at the regression of `diameter` to subset features, it supports the `MAPE` results. `H` accounts for `76.74%` of the predicting power for `diameter`. The remaining features have low, if any, direct correlation to `diameter`.

In [31]:
```
cfg = load_config("config.yaml")
df_model = pd.read_csv(cfg["paths"]["dataset_cleaned"])

features_all = ['a', 'e', 'i', 'H']
X = df_model[features_all].astype(float)
y = df_model[cfg["features"]["target_col"]].values.ravel()

mi_scores = mutual_info_regression(X, y, random_state=42)
df_mi = pd.Series(mi_scores, index=features_all).sort_values(ascending=False)

print("MUTUAL INFORMATION GEOMETRIC ANCHOR REPORT:")
print("=" * 50)
for feat, score in df_mi.items():
    print(f"Feature '{feat}': Mutual Information Score = {score:.4f}")
print("=" * 50)
```

```
MUTUAL INFORMATION GEOMETRIC ANCHOR REPORT:
==================================================
Feature 'H': Mutual Information Score = 0.7674
Feature 'a': Mutual Information Score = 0.2201
Feature 'e': Mutual Information Score = 0.0488
Feature 'i': Mutual Information Score = 0.0269
==================================================
```

Reviewing the sensitivity figure, it is clear that `H` is necessary to predict `diameter`. Error rates double when `H` is removed as can be seen by `RF` which had a `MAPE` increase from `14.44%` to `49.47%` when `H` was removed.

In [32]:
```
df_logs = pd.read_csv('logs/prediction_results.csv')
vutils.launch_interactive_plot(df_logs, plot_type='scarcity', title='Algorithmic Sensitivity to Feature Sc
```

```
HTML(value='<div>Stacked Informational Drop Analysis Rendered Successfully.</div>')
Output()
HBox(children=(ToggleButtons(description='Scale:', options=('Log Scale', 'Linear Scale'), value='Log Scal
e'),)…
```

The linear-to-logarithmic visualization of predicted versus actual sizes confirms that the dataset is heavily skewed, validating the operational necessity of the target logarithmic transformation. The figure demonstrates that while all six architectures successfully handled predictions within the `Medium` range, they struggled when handling `outliers` in the `Massive` range.

Classical models like `Random Forest` (`RF`) handled the scale boundaries the most effectively, based on how tightly their predictions cluster along the ideal identity fit line. Conversely, the `Wide & Deep` (`W&D`) network performed relatively well with `Massive` `outliers` but degraded severely when predicting in the `Sub-Kilometer` range. Because `W&D` architectures are structurally optimized to capture broad, population-wide averages, the combination of extreme dataset skewness and the physical inconsistency of small asteroids to reliably reflect light prevented the linear wide path from establishing effective rules to reliably predict `diameter`.

In [33]:
```
vutils.launch_interactive_plot(df_summary, plot_type='scatter', pred_col='Full Pred (km)', title='Logarithr
```

```
HTML(value="<div style='background-color: #f8f9fa; padding: 15px; border-radius: 6px; font-family: sans-ser
if;…
Output()
HBox(children=(ToggleButtons(description='Scale:', options=('Log Scale', 'Linear Scale'), value='Log Scal
e'),)…
```

The model benchmark figure supports that the classical models like `RF` and `GBR` perform better than the neural network models. All models appear to overpredict on `Sub-Kilometer` asteroids and underpredict on `Massive` asteroids. The tree-based models struggle with predicting larger asteroids because they are unable to predict `diameters` that exceed what they received during training. This can be seen in `RF` where `Ceres` and `Pallas` were predicted to be similar in size on the upper outlier even though `Ceres` is hundred of kilometers larger. This also supports tree-based models better accuracy because they are step-based and just split data base on rules. Neural networks tend to want to fit data to a pattern. Since the dataset is highly skewed and does not have a linear or reliable pattern, it makes tree-based models more accurate at predicting `diameter`.

In [34]:
```
vutils.launch_interactive_plot(df_summary, plot_type='bar', pred_col='Full Pred (km)', title='Multi Model I
```

```
HTML(value="<div style='background-color: #f8f9fa; padding: 15px; border-radius: 6px; font-family: sans-ser
if;…
Output()
HBox(children=(ToggleButtons(description='Scale:', options=('Log Scale', 'Linear Scale'), value='Log Scal
e'),)…
```

In [35]:
```
vutils.launch_interactive_plot(df_summary, plot_type='lines', pred_col='Full Pred (km)', title='Full Model
```

```
HTML(value="<div style='background-color: #f8f9fa; padding: 15px; border-radius: 6px; font-family: sans-ser
if;…
Output()
HBox(children=(ToggleButtons(description='Scale:', options=('Log Scale', 'Linear Scale'), value='Log Scal
e'),)…
```

# IV. REPORT & V. ACT

## 1. Hypothesis Testing

**Acceptance of Statistical Alternative Hypothesis ($H_{1,stat}$):** $\Delta R^2_{test} > 0$

- *Definition:* The inclusion of Group 2 features yields an increase in the out-of-sample $\Delta R^2_{test}$, indicating that the expanded features **do** increase the accuracy of predicting the `target` `diameter` (`Y`).
- *Result:* All six models resulted in a $\Delta R^2_{test}$ of no less than `0.4917`. This exceeded the statistical threshold of zero (`0`). This means the expanded features did increase the accuracy of predicting the `target` `diameter` (`Y`).

**Acceptance of Operational Null Hypothesis ($H_{0,ops}$):** $\Delta R^2_{test} > \delta$

- *Definition:* The inclusion of Group 2 features yield an operationally significant increase in accuracy to predict `target` `diameter` (`Y`), indicating that the restricted feature subset **is not** sufficient enough for operational use.
- *Result:* All six models resulted in a $\Delta R^2_{test}$ of no less than `0.4917`. This exceeded the operational threshold of `0.05`. This means the restricted feature subset is insufficient for production deployment.

## 2. Feature Variance Analysis

- Feature importance and sensitivity metrics confirm that `absolute magnitude` (`H`) accounts for `76.74%` of the variance explained when predicting `diameter`. Omitting `H` from the training pipeline more than triples the error rate across all models, demonstrating that the geometric elements (`a`, `e`, `i`) lack sufficient standalone signal and `H` is required to reliably and accurately predict `diameter`.

## 3. Model Performance & Selection

- **Final Model Selection:** `Random Forest` (`RF`)
  - `RF` is formally selected as the most reliable deployment model for predicting `asteroid diameter`. Across five-fold cross-validation on the full feature set, `RF` achieved a Mean $R^2$ of `0.8508` and an `out-of-sample` Mean `MAE` of `1.0443 km`. While `Gradient Boosting Regression` (`GBR`, `XGBOOST` in code) demonstrated a slightly higher Mean $R^2$ of `0.8548` and a comparable Mean `MAE` of `1.0360 km`, `RF` was ultimately selected for production deployment because it resulted in the lowest overall average `Mean Absolute Percentage Error` (`MAPE`) of `14.44%` compared to the average `MAPE` for `GBR` of `20.80%`.

## 4. Classical Modeling Insights

- Classical tree-based machine learning models performed the best for highly skewed asteroid data. This is likely because it utilizes discrete, step-based data partitioning to isolate numeric subgroups for predicting `diameter`. Tree-based models are limited because they can only predict within the range of their training data but were able to establish more reliable patterns in that range for predicting `diameter`. For instance, while `GBR` handled extreme values better at the `Massive` (>50km) scale (`8.10%` vs `18.03%` `MAPE`), `RF` outperformed it at the highly populous `Sub-Kilometer` (<1km) scale (`20.37%` vs `35.98%` `MAPE`), securing the lowest global error average.

## 5. Neural Network Insights

- Neural networks struggled to reliably predict `diameter` because they favor optimizing for generalized pattern fitting. The highly skewed dataset and inconsistent measurements of features like `H` were likely over-smoothed. This caused the models to overpredict at the `Sub-Kilometer` scale and struggle to handle the `Massive` scale. Furthermore, removing `H` caused complete mathematical failures during training. Without this feature, both the `W&D` and `GRN` models completely broke down on certain `asteroid` scales, dropping to negative accuracy levels and throwing massive errors exceeding $10^{21}$ kilometers.